NOTIONAL AND UNCLASSIFIED · BUILT FROM PUBLIC SOURCES ONLY

Securing a small UAS, from threat model to boot evidence

An overview of the UAS security portfolio for engineers new to secure boot, measured boot and TPMs: the architecture (P1), verified boot, measured boot and key sealing (P2), and the roadmap for P3 to P6

Numbers and statuses in this document are generated at build time from P1's data at commit `e539aa6` and from a pinned snapshot of P2 at commit `db31c14`. Nothing here is based on a real program or system. All keys mentioned are development or test keys; no private key appears in either repository.

Contents

About this document 3

1 P1: the system, its threats and its requirements 5
- 1.1 The system 5
- 1.2 Protecting command and control 6
- 1.3 Threats 7
- 1.4 Requirements and verification events 8
- 1.5 Why P2's requirements exist 8
- 1.6 How P1 checks itself 9

2 P2: verified boot, measured boot and key sealing 11
- 2.1 What P2 must show 11
- 2.2 Concepts 12
- 2.3 The P2 design 19
- 2.4 Test cases 24
- 2.5 Reading the evidence 31
- 2.6 Open items 35
- 2.7 Rebuilding and rerunning the tests 36

3 Roadmap: P3 to P6 40
- 3.1 P3: PKI, key management and signed updates; SITL harness 40
- 3.2 P4: Hardened CC image, MSS pipeline and supply chain 42
- 3.3 P5: Hardware and firmware security assessment 43
- 3.4 P6: RMF-as-code (OSCAL): configuration and procedure evidence 44

4 Limitations and open findings 45
- 4.1 What the audit did 45
- 4.2 Index of findings 46
- 4.3 Findings 47

Appendix: glossary and sources 57

About this document

This document explains the portfolio from first principles. It is written for an engineer who is new to secure boot, measured boot and Trusted Platform Modules (TPMs) and who must be able to understand the design, check the evidence, and rebuild and rerun the tests.

The portfolio is a set of repositories about one notional small unmanned aircraft system (UAS):

- **P1** defines the system, its threats, its security requirements and the plan for verifying them. It is the anchor for every other repository.
- **P2** produces the evidence for the P1 requirements on verified boot, measured boot and TPM key sealing on the companion computer, and signed firmware on the PX4 flight controller.
- **P3 to P6** are planned. Each owns other P1 requirements; the roadmap chapter lists them. P1 also names a P7, which fuzzes the companion computer's command filter (P1 `docs/04-architecture.md` DD-04); it owns no P1 requirement.

How to read it

The P1 chapter (chapter 1) explains the system and why each P2 requirement exists. The P2 chapter (chapter 2) first explains every concept P2 relies on, each before it is used, then the design, every test case, how to read the evidence, and how to rebuild everything. The roadmap (chapter 3) covers P3 to P6. The next chapter (chapter 4) reports the limitations and open findings of an independent audit of P1 and P2, reproduced unchanged from the audit's findings file. The appendix (Appendix: glossary and sources) holds a glossary and the sources.

If you only want to rebuild and rerun the tests, go to §2.7 and look up concepts in §2.2 as you meet them.

Conventions

- **Generated, never retyped.** Every table of requirements, threats, verification events, test cases and evidence runs, and every count or status in the text, is generated when the PDF is built: from P1's `data/*.csv` and from the P2 snapshot in P1's `overview/p2-snapshot/`. That snapshot is a byte-for-byte copy of P2's `data/requirements.csv`, `data/trace.csv`, `data/testcases.csv`, `data/p1/pin.txt` and the `summary.json` of each evidence run the test cases cite, at the P2 commit in `overview/p2-snapshot/pin.txt`. P1's CI checks the copy against P2 with `tools/check_p2_snapshot.py`. A number that is not generated is quoted from a named file at the stated commit.
- **Citations.** "P1 `docs/04-architecture.md` §4.2" means that section of that file in the P1 repository at the commit in the table below; "P2 `tools/predict_pcrs.py`:225" means line 225 of that file in P2 at the snapshot commit. Facts about U-Boot, PX4, NXP and the TCG specifications are cited through the P2 document that checked them, with P2's source tags such as [UBOOT] or [PX4-BL] (P2 `docs/references.md`).
- **Labels.** *Assumption* marks something accepted for analysis but not shown. *Emulated* means a run on QEMU with a software TPM, not on the hardware; what each emulated run stands in for is stated where it appears. *Planned* means a test case has not run.
- **Status words.** A test case runs in CI, on hardware, or both; "–" means it does not run there. P2 rolls test cases up into a status per verification event: *failed* if any run has failed, otherwise *passed* only when every applicable CI and hardware run has passed, *in progress* when some have, *planned* when none have (P2 `tools/render_views.py`, `rollup()`). P1 records an event as passed only when a P1 pull request links the evidence (P2 `docs/06-verification.md` §6.5).
- **Identifiers.** `SR-###` is a P1 security requirement, `SR-###.#` a P2 child requirement, `THR-###` a P1 threat, `VE-##` a P1 verification event, `TC-##` a P2 test case, `DD-##` and `DD2-##` design decisions in P1 and P2, `A-##` and `A2-##` assumptions, `OI-##` and `OI2-##` open items.

Built from

What	Commit or run
P1 tree the document was built from	`c361ea5964dfc3a31e6b04b51e9d229e5af0c718`
P1 data (`data/*.csv`): last commit that changed it	`e539aa638d3e44c1982341e927d974b4396e0576`
P2 snapshot (`overview/p2-snapshot/pin.txt`)	`db31c1451c7af859ae08cc2d4b073980cc666a1a` (samyoon727ca/uas-sec-p2-verified-boot)
P1 commit that P2's own snapshot pins (`data/p1/pin.txt` in P2)	`e539aa638d3e44c1982341e927d974b4396e0576` (the same P1 data as this document)
P2 evidence `evidence/ve07-ci/20261013-d9149f6/`	VE-07: GitHub Actions run 38349138261, code commit `d9149f6`, 2026-10-13T05:05:52Z
P2 evidence `evidence/ve08-ci/20261013-aac4299/`	VE-08: GitHub Actions run 38351585198, code commit `aac4299`, 2026-10-13T05:35:39Z

Toolchain that rendered this PDF: Node.js v22.23.3; Chrome/154.0.8037.97; markdown-it 14.3.2; @mermaid-js/mermaid-cli 12.0.0 with mermaid 12.1.0; puppeteer 25.12.0.

1 P1: the system, its threats and its requirements

P1 is the security architecture of a notional, unclassified small UAS built from open-source parts: a PX4 flight controller, a Linux companion computer, a ground control station and a MAVLink command link (P1 `README.md`). It sets down the system, the threats to it, the requirements that answer those threats, and a verification event for every requirement. Every later repository produces evidence for some of those requirements.

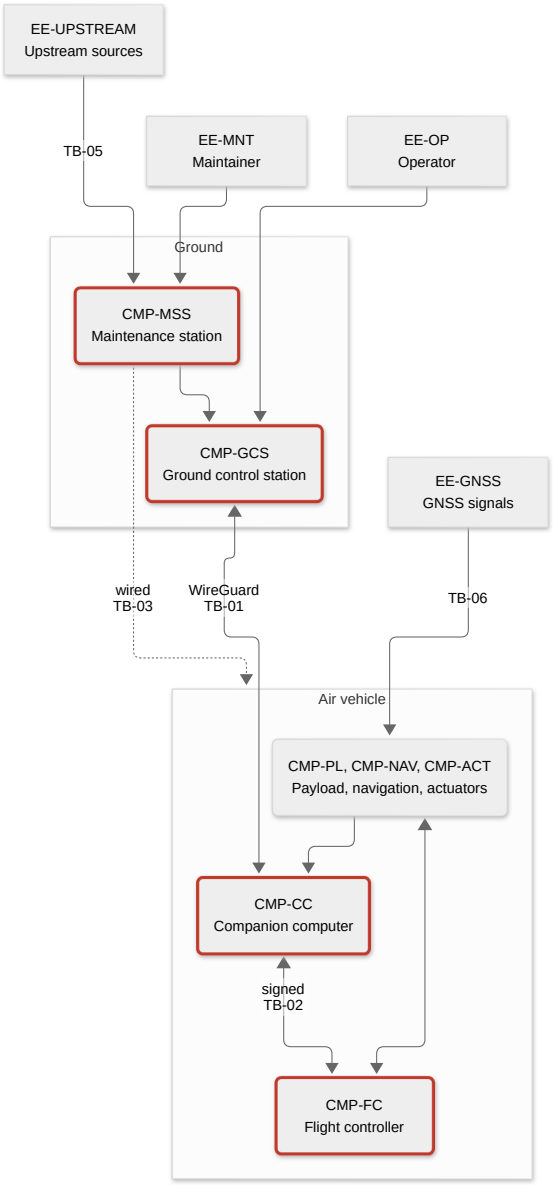
P1 keeps its facts in five CSV files under `data/`, which are the single source of truth. Python tools check them and generate the tables in P1's documents (P1 `data/README.md`). This chapter's tables are generated the same way, from P1's data at commit `e539aa6`.

1.1 The system

The vehicle flies a pre-planned survey pattern within radio line of sight, stores electro-optical imagery on board, streams video to the operator and returns; imagery is offloaded over a wired link after landing (P1 `docs/02-conops.md` §2). Its parts (P1 `docs/01-system-description.md` §1.2):

ID	Component	Basis	Role in this document
CMP-FC	Flight controller (FC)	PX4 on NuttX, Pixhawk-standard flight management unit	Flies the vehicle. Verifies MAVLink signatures. P2 makes it boot only signed firmware
CMP-CC	Companion computer (CC)	ARM64 Linux single-board computer	Terminates the encrypted link from the ground, filters commands, runs autonomy and payload. P2 makes it boot only a signed chain and keep its keys in a TPM
CMP-GCS	Ground control station (GCS)	QGroundControl on a Linux laptop	Where the operator plans and commands
CMP-MSS	Maintenance and support station (MSS)	Linux workstation, wired connections only	Builds, signs and loads software; provisions keys
CMP-RAD-A, CMP-RAD-G	Data-link radios	Generic IP radios	Untrusted transport: no credit is taken for their own encryption (assumption A-05)

The architecture's first design decision, DD-01, connects the radio to the CC and the CC to the FC, so every command-and-control (C2) path runs ground station → CC → FC (P1 `docs/04-architecture.md` §4.4). The diagram below is rendered from P1's own source, so it cannot drift from it; only its direction is changed, from left-to-right to top-to-bottom, to fit the page.

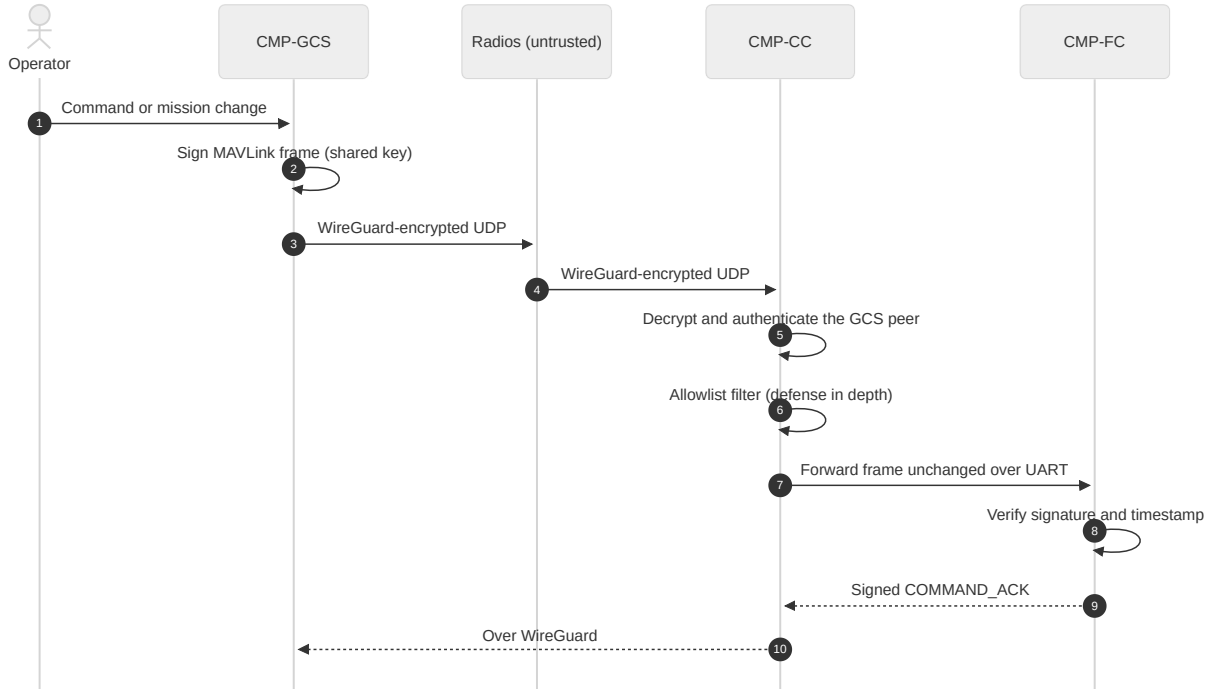


System context (P1 `docs/04-architecture.md` §4.1). TB-## labels are trust boundaries: TB-01 the air-ground radio link, TB-02 between the flight-critical FC and the CC, TB-03 the vehicle's physical boundary (storage and ports), TB-05 the supply chain, TB-06 the GNSS signal environment (P1 `docs/04-architecture.md` §4.1, trust boundaries table). Red outlines mark the holders of the shared MAVLink signing key.

1.2 Protecting command and control

Three layers protect commands from the ground (P1 `docs/04-architecture.md` §4.3):

1. **A WireGuard tunnel from GCS to CC** (DD-02) gives confidentiality and peer authentication across the radio link. The CC accepts nothing else on its radio interface (requirement SR-001).
2. **An allowlist filter on the CC** (DD-04) limits which MAVLink messages and commands reach the FC. It is defense in depth, not an authentication point.
3. **MAVLink message signing, GCS to FC.** Each MAVLink 2 frame carries a 48-bit signature computed with SHA-256 over a 32-byte secret key and the frame (P1 `docs/01-system-description.md` §1.3). This end-to-end signing is design intent: it has not yet been shown to work between the pinned QGroundControl and PX4 versions (open item OI-01, closed by verification event VE-02 in P3).



The command path (P1 `docs/04-architecture.md` §4.3). Steps 2 and 8, signing and verification, are design intent until VE-02 passes.

The weak point is the signing key. PX4 uses one shared signing key for every node: the FC, the CC, the GCS and the MSS all hold the same 32-byte secret (P1 `docs/04-architecture.md` §4.2). Two consequences follow, and they are the reason P2 exists:

- **Any key holder can forge any command.** Code running on the CC, or a copy of the CC's key, gives full control of the vehicle: parameter writes, mission upload, arming, flight termination (P1 `docs/04-architecture.md` §4.2, item 1).
- **The CC is the most exposed holder.** It terminates the tunnel, parses traffic from the radio link, runs general-purpose Linux and is powered throughout flight (§4.2, item 2). Protecting it is the job of P2 (what it boots, where its keys live) and P4 (the hardened image).

1.3 Threats

P1 analyzes threats with **STRIDE per element**. Each element of the data-flow diagram (process, external entity, data store, data flow) is examined for the six STRIDE categories that apply to its type: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service and **E**levation of privilege (P1 `docs/05-threat-model.md` §5.1). Each threat gets a mission-impact rating from MI-1 (loss of vehicle control, or a safety hazard) to MI-4 (negligible). The rating is the consequence if the threat is realized, scored before mitigation, with no likelihood term; the scale is project-defined (assumption A-13; P1 `data/README.md`).

The adversary has three tiers (assumption A-08): (1) RF-proximate, with a software-defined radio, able to jam, eavesdrop and inject; (2) physical access to a lost or captured vehicle or unattended ground equipment; (3) able to insert code through an upstream source or a build host. Tier 2 matters most for P2: a captured vehicle gives an adversary time with its storage, its ports and its circuit board.

In numbers: **66 threats** on 40 STRIDE elements; by impact 42 MI-1, 16 MI-2, 6 MI-3, 2 MI-4; 63 mitigated by requirements and 3 accepted with a written rationale. Most MI-1 threats end the same way: an adversary holds the shared signing key, or runs code on a node that holds it (P1 `docs/05-threat-model.md`, introduction). The threat P1 ranks first is THR-010, "Code execution on the CC yields full vehicle C2" (P1 `docs/05-threat-model.md` §5.3).

1.4 Requirements and verification events

P1 derives **45 requirements**, each a single "shall" (P1 `docs/07-requirements.md` §7.1). By component: CC 15, MSS 13, FC 9, GCS 8. By verification method: Test 24, Inspection 13, Demonstration 8. By the repository that will produce the evidence: P2 3, P3 14, P4 14, P5 2, P6 12. P1's validator (`tools/validate_trace.py`) enforces that every requirement traces to at least one threat (through 108 links in `data/trace.csv`), is allocated to exactly one component, and is covered by a verification event with the same method and evidence repository.

There are **17 verification events**; their statuses in P1 are: 17 planned. An event's status changes from planned only through a P1 pull request that links its evidence, pinned to a commit in the repository that produced it (P1 `data/README.md`, `verification.csv`).

Evidence repo	Requirements	Verification events	Event status in P1
P2	3	VE-07, VE-08	2 planned
P3	14	VE-01, VE-02, VE-03, VE-04, VE-05, VE-06, VE-17	7 planned
P4	14	VE-09, VE-10, VE-11	3 planned
P5	2	VE-12, VE-13	2 planned
P6	12	VE-14, VE-15, VE-16	3 planned

Requirements and verification events per evidence repository, generated from P1's `data/requirements.csv` and `data/verification.csv`.

1.5 Why P2's requirements exist

P1 assigns 3 requirements to P2, SR-005, SR-010, SR-018, verified by VE-07, VE-08. Their parent threats are the reason for each one.

ID	Requirement	On	Method	Verified by	Parent threats
SR-005	The CC shall boot only boot-chain components signed with the project boot key, verified from a hardware root of trust.	CC	Test	VE-07	THR-008, THR-010, THR-032
SR-010	The CC shall seal its WireGuard private key and its copy of the signing key to its TPM, bound to the measured boot state.	CC	Test	VE-07	THR-007, THR-034, THR-035
SR-018	The FC shall boot only firmware signed with the project firmware key, using PX4 Bootloader Secure Boot.	FC	Test	VE-08	THR-030, THR-041, THR-058

Threat	Title	Element	STRIDE	Impact	P2 requirement
THR-007	Vehicle impersonated with a stolen CC WireGuard key	CMP-CC Companion computer	S	MI-2	SR-010
THR-008	Persistent modification of CC software	CMP-CC Companion computer	T	MI-1	SR-005
THR-010	Code execution on the CC yields full vehicle C2	CMP-CC Companion computer	E	MI-1	SR-005
THR-030	FC firmware or parameters modified through the bootloader or debug port	DS-FC-FW FC firmware and parameters	T	MI-1	SR-018
THR-032	CC storage rewritten offline	DS-CC-SYS CC system image and configuration	T	MI-1	SR-005
THR-034	Keys extracted from a captured CC	DS-CC-CRED CC credentials	I	MI-1	SR-010
THR-035	CC trust anchors replaced	DS-CC-CRED CC credentials	T	MI-1	SR-010
THR-041	Release artifacts tampered before loading	DS-MSS-ART MSS release artifacts	T	MI-1	SR-018
THR-058	Unsigned firmware loaded through the USB bootloader	DF-11 FC maintenance load	T	MI-1	SR-018

The P1 threats that P2's requirements answer: 9 threats, 8 of them rated MI-1.

SR-005, verified boot of the CC. THR-008 is a compromise that survives reboot by modifying the CC's boot chain; THR-032 rewrites the CC's storage offline, which a tier-2 adversary with a captured or unattended vehicle can do by removing the SD card. Either way the CC would run attacker code with the signing key and the WireGuard key in reach, which is THR-010. Verified boot means the CC refuses, before running it, any boot-chain component not signed with the project's boot key. P1's assumption A-04 says where the trust starts: the CC's system-on-chip (SoC) has secure boot, whose boot ROM checks the first boot-loader stage against a key hash fused into the chip (P1 `docs/03-assumptions.md`). The concepts behind this are explained in §2.2.

SR-010, keys sealed to the TPM. THR-034 extracts the CC's copy of the signing key and its WireGuard private key from a captured vehicle; THR-007 uses the stolen WireGuard key to pose as the vehicle; THR-035 replaces the CC's stored trust anchors. Sealing means the keys are stored encrypted by a TPM and released only when the TPM's record of the boot (its measurements) matches an approved state. Removed storage holds only ciphertext that no other TPM can open, and a modified boot chain cannot get the keys. A-04 is explicit that the TPM measures and seals but does not verify: it cannot stand in for secure boot. THR-034 keeps a residual risk after sealing: with time and tools, an adversary can attack the bus between the SoC and the TPM, or replay approved measurements if the TPM can be reset on its own (P1 `docs/05-threat-model.md` §5.5). Physical attacks are P5's scope.

SR-018, signed FC firmware. THR-058 loads unsigned firmware through the PX4 USB boot loader, which accepts any image unless secure boot is enabled; THR-030 modifies firmware with physical access; THR-041 tampers with release artifacts between build and load. PX4 Bootloader Secure Boot checks an Ed25519 signature before starting the firmware. Its root of trust is the boot loader in the FC's flash, which sets no write-protection, so SR-018 holds only while SR-024 (P5) keeps the debug port closed (SR-018's rationale in P1 `data/requirements.csv`). It does not stop an *older* signed release from booting: P2 found that gap, and P1 recorded it as THR-066, "Older signed FC firmware loaded through the USB bootloader", mitigated by SR-045 in P3 (P2 `docs/01-scope-and-trace.md` §1.4, change 2).

The two verification events P2 runs, as P1 defines them:

Event	Title	Method	Verifies	P1 status	Pass criteria
VE-07	CC verified boot and key sealing	Test	SR-005, SR-010	planned	Only the signed chain boots; the keys unseal only in the measured, unmodified boot state.
VE-08	FC boots only signed firmware	Test	SR-018	planned	Only the signed image starts PX4; the others stay in the bootloader.

Event	Procedure in P1's data/verification.csv
VE-07	On the dev board: boot a signed chain; then boot with a modified bootloader, kernel and boot configuration in turn; then try to unseal the CC keys from a modified boot state and from removed storage.
VE-08	Enable PX4 Bootloader Secure Boot; flash signed, unsigned and modified-signed firmware over USB.

P2 splits these two events into 14 test cases (§2.4). P1's status is *planned* for VE-07 and *planned* for VE-08; it changes only when every CI and hardware test case of the event has passed and a P1 pull request records it.

1.6 How P1 checks itself

Every push to P1 runs four CI jobs (P1 `.github/workflows/validate.yml`):

- **traceability:** unit tests, then `tools/validate_trace.py`, which fails if a threat is neither mitigated nor accepted, a requirement has no parent threat or no verification event with the same method and repo, an identifier is orphaned, a framework ID is not in its pinned catalog, or an event is marked passed or failed without evidence pinned to a commit (P1 `README.md`, "Verification evidence"). Then `tools/render_views.py --check` and `tools/gen_sysml.py --check`, which fail if a generated document table or the generated SysML v2 model has drifted from the CSVs.
- **sysml:** validates the SysML v2 model with the OMG Pilot Implementation.
- **brief:** builds the eleven-slide brief PDF with a pinned toolchain.

- **overview:** checks the P2 snapshot against P2 and builds this document (`overview/README.md`).

§2.7 runs the Python checks on a clean machine, with their expected output.

2 P2: verified boot, measured boot and key sealing

P2 (`samyoon727ca/uas-sec-p2-verified-boot`) produces the verification evidence for the P1 requirements of the previous chapter: the companion computer boots only a signed chain verified from a hardware root of trust (SR-005); its keys are sealed to its TPM and released only to the measured boot state (SR-010); the PX4 flight controller boots only signed firmware (SR-018). It does so through P1's verification events VE-07 and VE-08, split into 14 test cases (P2 `README.md`).

This chapter states P2 as of commit `db31c14`. Where it is now:

Event	Verifies (P1)	CI runs passed	Hardware runs passed	P2 status	P1 status
VE-07 CC verified boot and key sealing	SR-005, SR-010	9 of 9	0 of 9	in progress	planned
VE-08 FC boots only signed firmware	SR-018	1 of 1	0 of 3	in progress	planned

Generated from the P2 snapshot (`data/testcases.csv`) with P2's own roll-up rule, and from P1's `data/verification.csv` for the P1 status.

In CI, 10 of 10 test cases have passed, in emulation or on the host; on hardware, 0 of 12. The companion-computer board, a DEBIX Model A (NXP i.MX 8M Plus), is on order (P2 `docs/02-cc-boot-chain.md`, status paragraph). P2's documents specify "an SPI TPM 2.0 module on ECSPI2" but name no module (P2 `docs/02-cc-boot-chain.md` §2.1); according to the project owner, the module has not been chosen yet. P2's rule is that nothing is called working until its test case passes (P2 `docs/06-verification.md`), and this document follows it: an emulated pass is described as exactly that.

2.1 What P2 must show

P2 refines a P1 requirement only where it cannot be tested as written. A child requirement keeps its parent's component and verification method and traces only to its parent's P1 threats; a threat P1 lacks goes back to P1 as a proposed change instead (P2 `docs/01-scope-and-trace.md` §1.2). There are 5 children:

ID	Requirement	Parent	Parent threats	Verified by
SR-005.1	The CC shall take its boot-loader configuration only from the verified boot-loader image and from signed FIT configurations.	SR-005	THR-008, THR-032	TC-01, TC-04
SR-010.1	The CC shall bind its sealed keys to SHA-256 PCRs 0, 1, 8 and 9 as extended by the boot loader.	SR-010	THR-007, THR-034	TC-05, TC-06, TC-10
SR-010.2	The CC shall unseal its keys only under a PCR policy signed with the MSS policy key.	SR-010	THR-034	TC-05, TC-08
SR-010.3	The CC shall refuse to unseal its keys under a policy whose epoch is below the minimum epoch recorded in its TPM.	SR-010	THR-034	TC-08
SR-010.4	The CC shall unseal its keys only over a salted TPM session with response encryption.	SR-010	THR-034	TC-09

P2's child requirements, generated from the P2 snapshot (`data/requirements.csv`, `data/trace.csv`, `data/testcases.csv`).

SR-018 has no child. Its gaps were P1-level, so P2 proposed them to P1, which adopted them: THR-066 (older signed firmware) with SR-045, and the dependence of SR-018 on SR-024 (P2 `docs/01-scope-and-trace.md` §1.4).

P2 also works under six assumptions of its own (P2 `docs/01-scope-and-trace.md` §1.3). Each is accepted for analysis, not shown:

ID	Assumption (summarized)	Why it matters
A2-01	Every `u-boot.itb` ever signed with the root key has measured boot enabled	A signed U-Boot that does not measure leaves the PCRs at reset values; with a verification flaw, the kernel it started could extend the approved values itself
A2-02	The TPM resets only at power-on; its reset pin is not driven by software (DD2-17)	A TPM reset on its own returns its PCRs to zero, so approved measurements could be replayed. P5 covers physical attack
A2-03	In emulation, QEMU loading `u-boot.bin` and its control devicetree stands in for the hardware root of trust	Emulation cannot show a modified boot loader being refused; TC-02 is hardware only
A2-04	swtpm 0.7.3 implements the TPM 2.0 commands P2 uses as the TCG specifies	swtpm is not a certified TPM and keeps its state in a host file; the hardware runs repeat the sealing cases
A2-05	PX4 Bootloader Secure Boot behaves the same at `v1.18.0` as at `v1.18.0-rc1`	Rechecked at the PX4 re-pin (P1 DD-06)
A2-06	U-Boot's ECSPI driver can drive the chosen SPI TPM on the DEBIX	Measured boot on the hardware depends on it (OI2-01)

2.2 Concepts

This section explains every idea the P2 design uses, in the order the design needs them. Each subsection ends with how P2 uses the idea.

Hashes and digital signatures

A **cryptographic hash** such as SHA-256 maps any input to a fixed 32-byte digest. Changing one bit of the input changes the digest unpredictably, and finding two inputs with the same digest is infeasible. A digest is therefore a short, reliable name for exact content: if the digest of what you have equals a digest you trust, you have the trusted content.

A **digital signature** binds content to a key pair. The holder of the private key signs (a digest of) the content; anyone holding the matching public key can verify the signature, and nobody without the private key can produce one that verifies. A verifier therefore needs two things it can trust: the verification code, and the public key. Everything in secure boot is about where those two come from.

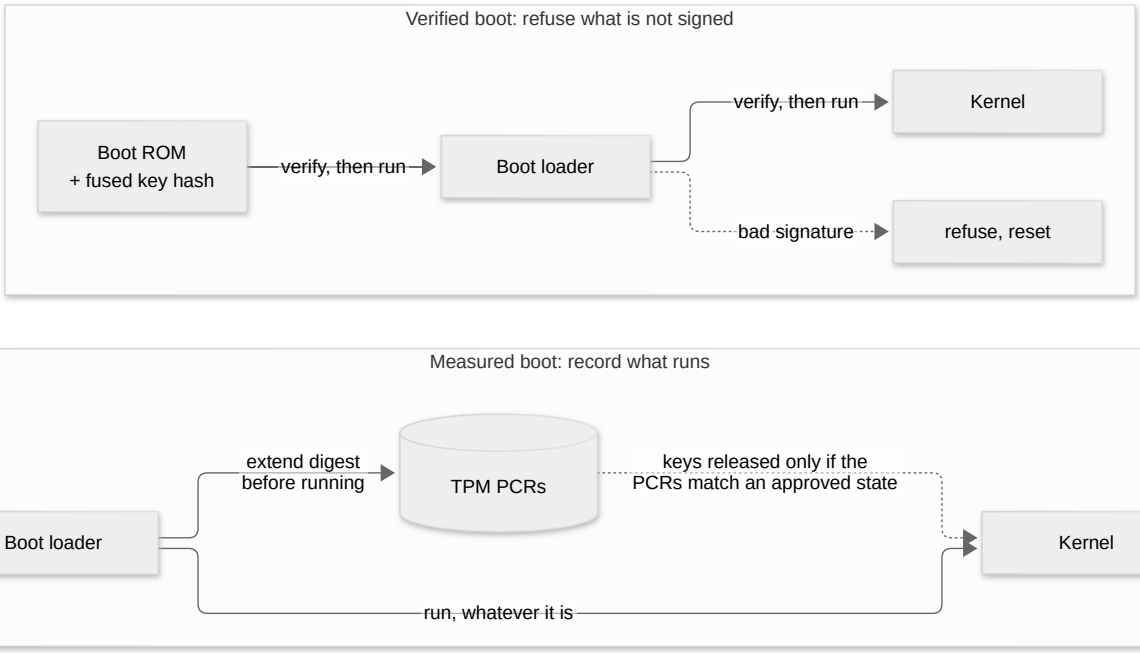
P2 uses four signature algorithms, each fixed by the verifier that checks it (P2 `docs/05-key-management.md` §5.1): RSA-4096 for the root boot key K-CC-ROOT (checked by the i.MX boot ROM), RSA-3072 for the OS key K-CC-OS (checked by U-Boot), ECDSA P-256 for the policy key K-CC-POL (checked by the TPM) and Ed25519 for the flight-controller keys K-FC-FW and K-FC-REC (checked by the PX4 boot loader). NIST SP 800-57 rates RSA-3072 and ECDSA P-256 at 128 bits of security strength; RSA-4096 falls between its 128-bit and 192-bit rows, so P2 counts it as 128 bits (P2 `docs/references.md`, [SP800-57]). P2 gives Ed25519 no strength figure, because its 253-bit group order falls in that table's 112-bit row while it is commonly described as 128-bit (P2 `docs/05-key-management.md` §5.1).

Roots of trust, verified boot and measured boot

Verified boot is a chain: each stage checks the signature of the next stage before running it, and refuses to run it if the check fails. A chain is only as strong as its first link, so the first check must be made by code and a key that an attacker cannot change. On a system-on-chip that is a **hardware root of trust**: verification code in mask ROM, which cannot be rewritten, and a public key (or its hash) burned into one-time-programmable fuses.

Measured boot is different. Each stage *records* a digest of the next stage in a TPM before running it, and runs it whatever it is. Nothing is refused; instead the TPM ends up holding an unforgeable summary of what booted. That summary can then gate a decision, such as releasing a key.

P1's assumption A-04 keeps the two apart: SoC secure boot verifies the chain (SR-005), and the TPM measures and seals (SR-010). A TPM cannot stand in for secure boot, because it stops nothing (P1 `docs/03-assumptions.md`).



Verified boot stops a modified chain from running. Measured boot lets it run but leaves evidence in the TPM that keeps the keys locked.

How P2 uses it: the i.MX 8M Plus boot ROM is the root of trust for verification (next subsection), U-Boot continues the chain, and U-Boot also measures what it boots into the TPM.

NXP High Assurance Boot (HAB) on the i.MX 8M Plus

HAB is the verified-boot feature in the boot ROM of NXP i.MX processors. Images are signed offline with NXP's Code-Signing Tool (CST), which produces a Command Sequence File (CSF) holding the commands for the ROM, a table of public keys, the signatures and the certificates (U-Boot `doc/imx/habv4/introduction_habv4.txt` at `v2026.07`, cited by P2 as [UBOOT-HAB]).

The keys at the top are **Super Root Keys (SRKs)**: a table of up to four public keys. The chip does not store the keys; it stores the SHA-256 hash of the SRK table in fuses. At boot the ROM hashes the table carried in the CSF, compares it with the fused hash, and only then uses the table's keys to check the signatures (P2 `docs/02-cc-boot-chain.md` §2.1; on the i.MX 8M the SRK hash lives in fuse bank 6 words 0–3 and bank 7 words 0–3, [NXP-FUSE]).

On the i.MX 8M Plus the ROM verifies the first-stage loader (SPL, with its DRAM training firmware). SPL then asks the ROM, through the ROM's HAB API, to verify `u-boot.itb`, which holds U-Boot, the Arm Trusted Firmware stage BL31 and U-Boot's own devicetree (P2 `docs/02-cc-boot-chain.md` §2.2). U-Boot's build signs both with the CST when `IMX_HAB` is set ([UBOOT]).

Two properties matter for P2:

- **Enforcement needs a closed chip.** Burning the SRK hash is the first irreversible step; the chip still boots unsigned images and only logs failed checks as "HAB events". Burning the SEC_CONFIG fuse closes the chip; only then does the ROM refuse unsigned or wrongly signed images. On an open chip U-Boot's HAB call even reports success when authentication failed ([UBOOT] `arch/arm/mach-imx/hab.c`, cited in P2 `docs/references.md`). A wrong SRK hash on a closed chip gives a board that does not boot. P2 takes each fuse step only with the owner's explicit go-ahead at that moment (OI2-02; P2 `docs/05-key-management.md` §5.2, rule 5).
- **The key table is fixed forever.** An SRK can be revoked by fuse, and the root can move to another of the four, but no key can be added: a new table needs a new chip (P2 `docs/05-key-management.md` §5.1).

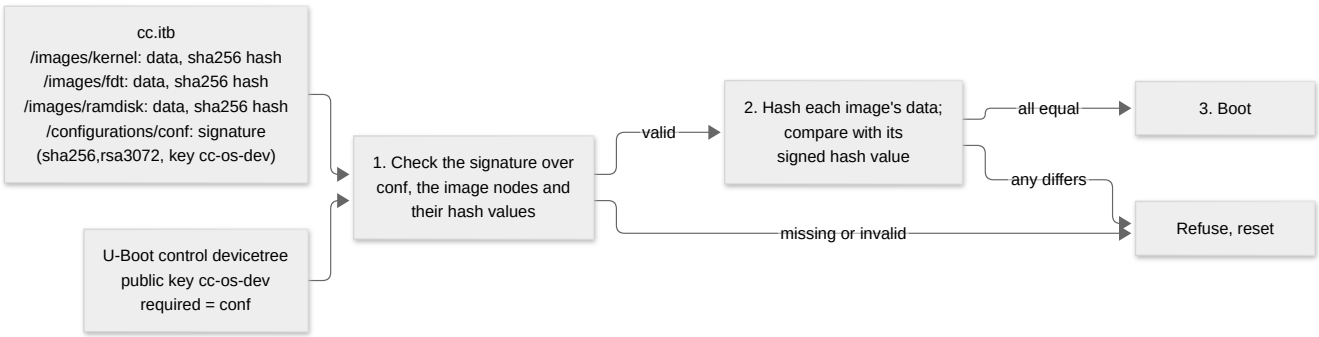
How P2 uses it: K-CC-ROOT is the table of RSA-4096 SRKs (DD2-16). There is no vendor-signed stage before the project's key: U-Boot itself is verified from hardware (P2 `docs/02-cc-boot-chain.md` §2.1, DD2-01).

U-Boot FIT images and configuration signing

U-Boot boots Linux from a **FIT** (Flattened Image Tree): one file, in the same binary format as a devicetree, holding several images (here the kernel, the initial RAM filesystem or *initramfs*, and the kernel's devicetree) and one or more *configurations* that name which images go together. Each image carries a hash subnode with its SHA-256 digest.

U-Boot can verify a signature over an image or over a configuration. When a **configuration** is signed, `mkimage` signs the configuration node, the image nodes it references with their hash subnodes, and the FIT's root structure. At boot U-Boot rebuilds that list of nodes from the configuration's own references, checks the signature over them, and then checks each image's data against its now-trusted hash (U-Boot `doc/usage/fit/signature.rst`, "Signed Configurations" and "Details", at `v2026.07`). Signing configurations rather than images stops a *mix-and-match* attack, in which signed images that were never released together are paired in a new FIT (P2 `docs/02-cc-boot-chain.md` §2.2, DD2-04).

The public key U-Boot verifies with lives in U-Boot's own control devicetree. Marked `required`, it makes any configuration without a valid signature fail: U-Boot prints `Failed to verify required signature` and does not boot it (P2 `docs/02-cc-boot-chain.md` §2.3).



How U-Boot checks a signed configuration (P2 `cc/fit/cc.its`; U-Boot `doc/usage/fit/signature.rst`). A changed byte in an image leaves check 1 passing and fails check 2; an unsigned, wrongly signed or added configuration fails check 1.

How P2 uses it: the CC's FIT has one configuration, signed with K-CC-OS (RSA-3072, SHA-256), covering the kernel, devicetree and initramfs (P2 `cc/fit/cc.its`). The K-CC-OS public key sits in the control devicetree inside the HAB-verified `u-boot.itb`, so the chain runs ROM → K-CC-ROOT → U-Boot with K-CC-OS → FIT.

Measured boot: PCRs, extend and the event log

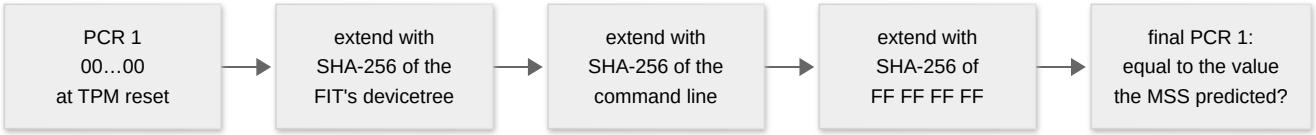
A TPM holds a set of **Platform Configuration Registers (PCRs)**. A PCR cannot be written; it starts at a fixed value at TPM reset (all zeros for the PCRs P2 uses) and has one operation, **extend**:

$$PCR_{new} = \text{SHA-256}(PCR_{old} \parallel \text{digest})$$

where ‖ is concatenation. Extending is one-way and order-sensitive, so the final value commits to the whole sequence of digests extended since reset. Nobody can reach a chosen value except by extending exactly the same digests in the same order from reset. P2's prediction tool implements extend as one line (P2 `tools/predict_pcrs.py`:225–230). A TPM keeps one set of PCRs per hash algorithm (a *bank*); P2's policy uses only the SHA-256 bank (P2 `docs/03-measured-boot-sealing.md` §3.1).

A PCR value alone does not say what was measured. The measuring software therefore also writes an **event log**: for each extend, the PCR index, an event type, the digest and a description. Replaying the log (extending each logged digest into zeroed registers) must give the PCR values; if it does, the log tells you exactly what was measured. U-Boot hands its log to Linux through the `linux,sml-base` and `linux,sml-size` properties of the TPM's devicetree node, and Linux exposes it as `/sys/kernel/security/tpm0/binary_bios_measurements` (P2 `docs/03-measured-boot-sealing.md` §3.1; `cc/initramfs/init`:17).

With `CONFIG_MEASURED_BOOT`, U-Boot `v2026.07` measures during `bootm`, after finding the images and before loading, decompressing or patching them, so every value can be predicted from the release files alone (P2 `docs/03-measured-boot-sealing.md` §3.1; `tools/predict_pcrs.py`:4–19):



PCR 1 as U-Boot builds it. PCR 0 gets the U-Boot version string and the separator, PCR 8 the kernel, PCR 9 the initramfs; PCRs 2 to 7 only the separator.

The events of the emulated chain, as P2's prediction tool computed them in the CI run behind TC-10 (from the evidence `summary.json` in the P2 snapshot):

PCR	Event type	Event data	SHA-256 of what was measured
0	EV_S_CRTM_VERSION	U-Boot 2026.07 (Jul 06 2026 - 23:50:43 +0000)	`460eda2cf7d6fb53…`
8	EV_COMPACT_HASH	linux	`6001f54164a5201c…`
9	EV_COMPACT_HASH	initrd	`e690e2cabc096334…`
1	EV_TABLE_OF_DEVICES	dts	`d742d34e1d64406b…`
1	EV_PLATFORM_CONFIG_FLAGS	console=ttyAMA0 rdinit=/init panic=-1	`bb889c886a92e92a…`
0–7	EV_SEPARATOR	ffffffff	`ad95131bc0b799c0…`

Policy PCR	Predicted SHA-256 value (equal to measured and replayed in TC-10)
0	`30c8a64625e906bd4cbb92442cbb825d80a4e91559aece4c5254e554a7fb54e0`
1	`a59fee377561a1ff23d7c9e6f1d6853f054269470832e5a0b0198c42f04e1dc6`
8	`872c40a29504ea646eae76dd60ff0a8078344d933f9c304cfeefb688ce0bb4e8`
9	`2e34bd575fc57407b450443791269d869fcd50fba28fa5352bc687aff9de8358`

How P2 uses it: the keys are bound to SHA-256 PCRs 0, 1, 8 and 9, which together identify the boot loader and everything it starts (SR-010.1). PCRs from 10 up change at runtime and are left to P4 (P2 `docs/03-measured-boot-sealing.md` §3.1). U-Boot ignores a failed measurement and boots anyway; the PCRs then hold no approved values, so the keys stay sealed. P2 calls this "fails open for boot, closed for keys".

TPM objects and sealing

A TPM 2.0 has keys that are generated inside it and never leave it in the clear. Objects are organized under **parents**: a storage key in the TPM's owner hierarchy can *wrap* a child object, producing a public part and a private part encrypted and integrity-protected under the parent. The wrapped object can be kept on ordinary storage; to use it, software **loads** it back into the TPM under the same parent. Only that parent, in that TPM, can unwrap it.

A **sealed data object** is a small secret wrapped this way. `TPM2_Unseal` returns the secret, but only if the caller satisfies the object's authorization. Every object has a **name**, the digest of its public area; names identify keys and objects in policies.

How P2 uses it (P2 `docs/03-measured-boot-sealing.md` §3.2):

- The secrets are the CC's WireGuard private key and its copy of the MAVLink signing key (P1's data store DS-CC-CRED).
- The parent is an ECC P-256 storage key, persistent at handle `0x81000001`. Its name is recorded at provisioning, and the unseal agent refuses any other parent.
- The sealed objects have the attributes `fixedtpm` (cannot be duplicated to another TPM) and `fixedparent`, and `adminwithpolicy` without `userwithauth`, so a policy is the only way to authorize them.
- The wrapped objects, the policy key's public key and each release's signed policy are stored on CC storage. None of it is secret, and it is useless away from this TPM. This is what TC-07 tests.

Policy sessions

A TPM object can be authorized by a password or by a **policy**. A policy is a digest built by chaining policy commands, each of which updates it as

$\text{policyDigest}_{\text{new}} = \text{SHA-256}(\ \text{policyDigest}_{\text{old}} \parallel \text{command code} \parallel \text{command-specific data}\ )$

starting from 32 zero bytes (P2 `tools/approve_policy.py`:118–130). When an object is sealed, the policy digest it requires is stored in it as its `authPolicy`.

To use the object, software opens a **policy session** in the TPM and runs the same policy commands. Each command checks its condition against the TPM's real state and folds the result into the session's digest. For example, `TPM2_PolicyPCR` folds in a digest of the *current* values of the selected PCRs; `TPM2_PolicyNV` checks the contents of a TPM non-volatile index and fails if the check fails. When the object is finally used, the TPM compares the session's digest with the object's `authPolicy`; only an exact match authorizes the use.

A policy made of `PolicyPCR` alone ties the keys to one set of PCR values. Any software update changes the PCRs, so every update would need the keys re-sealed, which means having them in the clear on a running system. The TCG calls this *PCR brittleness* (P2 `docs/03-measured-boot-sealing.md` §3.3, citing [TCG-P1] Figure 16).

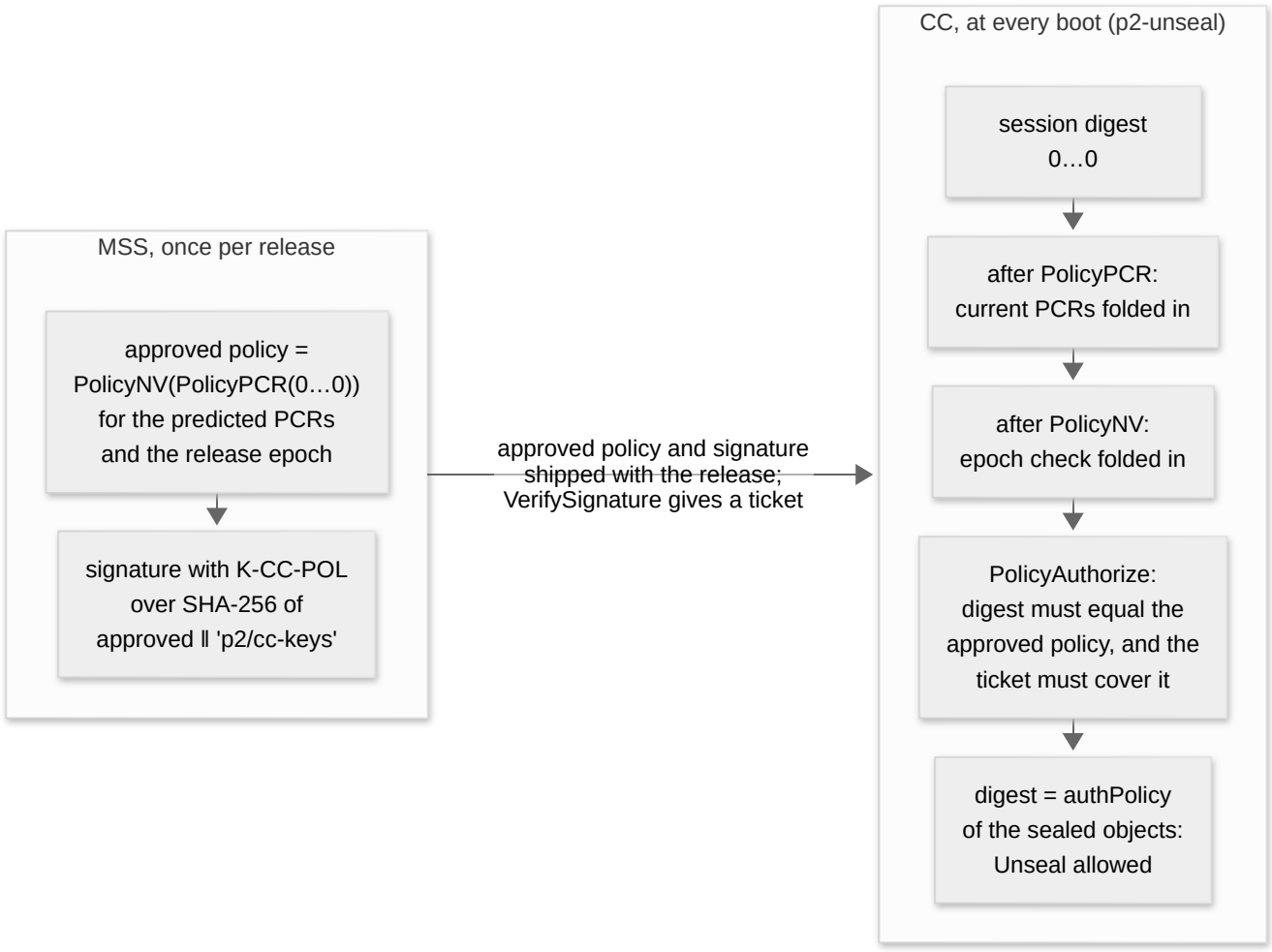
PolicyAuthorize: approval by signature

`TPM2_PolicyAuthorize` replaces "these exact PCR values" with "any state that the holder of this signing key has approved". The sealed object's `authPolicy` names the approving key and a reference string, not PCR values:

authPolicy = SHA-256(SHA-256(0…0 ‖ TPM_CC_PolicyAuthorize ‖ name of K-CC-POL) ‖ policyRef)

(P2 `tools/approve_policy.py`:128–130). To unseal, software:

1. runs the real conditions in a session (`PolicyPCR`, then `PolicyNV`), which leaves the session digest equal to some *approved policy* digest, if the state is the approved one;
2. asks the TPM to verify, with K-CC-POL's public key, a signature over SHA-256(approved policy ‖ policyRef), which yields a *ticket* proving the TPM checked it;
3. calls `PolicyAuthorize` with the approved policy, the policyRef, the key's name and the ticket. The TPM checks that the session digest equals the approved policy and that the ticket covers it, then replaces the session digest with the value that matches `authPolicy`.



The two halves of the release policy. The sealed objects never change; each release ships a new signed approval (P2 `docs/03-measured-boot-sealing.md` §3.3).

How P2 uses it (SR-010.2): the CC's keys are sealed once, to `PolicyAuthorize(K-CC-POL, "p2/cc-keys")`. For each release the MSS predicts the PCRs, builds the approved policy and signs it, so a legitimate update needs no re-sealing, and nothing on the CC can approve its own state. The policyRef `p2/cc-keys` names this use, so a policy the MSS signs for another purpose cannot be reused here; no test case checks this binding yet (“A-05: The policyRef binding is untested; emptying it leaves every case green” in §4.3). The K-CC-POL public key must be loaded into the owner hierarchy: a key loaded into the NULL hierarchy yields a NULL ticket, which `PolicyAuthorize` rejects (P2 `docs/03-measured-boot-sealing.md` §3.2–§3.3). A prototype found that swtpm 0.7.3 rejects an RSA-3072 key for `TPM2_LoadExternal`, which is why K-CC-POL is ECDSA P-256 (§3.7, DD2-11).

NV indices and the bit-field epoch

A TPM also has small areas of **non-volatile (NV) memory**, each an *NV index* with its own attributes and authorization. One NV type, `TPM_NT_BITS`, is a 64-bit field that starts at zero and can only gain bits: `TPM2_NV_SetBits` ORs bits in, and nothing clears them (P2 `docs/03-measured-boot-sealing.md` §3.4, citing [TCG-P1] §37.2.6.4).

`TPM2_PolicyNV` compares an NV index's contents (A) with an operand (B) given in the policy. With the operation `TPM_EO_BITCLEAR` it passes only if every bit set in B is clear in A, that is (A & B) = 0 (P2 `docs/references.md`, [TCG-P2] Table 18).

That gives **anti-rollback at key release** (SR-010.3). P2 keeps an *epoch* in the `TPM_NT_BITS` index at `0x01000001`, written once at provisioning with no bits set. Committing epoch *k* sets bit *k*. The approved policy for a release at epoch *E* uses as operand every bit above *E* (P2 `tools/approve_policy.py`:133–137), so it passes only while no newer epoch has been committed:

Bits set in the TPM	Policy of a release at epoch 0 (operand: bits 1 to 63)	Policy of a release at epoch 1 (operand: bits 2 to 63)
none (as provisioned)	passes	passes: a new release unseals before its epoch is committed
bit 1 (epoch 1 committed)	**fails at PolicyNV**: the older release can no longer unseal	passes

A bit field is used instead of a TPM counter because a counter's first increment lands above any value that index has held on that TPM, so its starting value differs per TPM and the MSS would have to sign per-device policies. A bit field starts at zero everywhere, so one signed policy works on every CC (P2 `docs/03-measured-boot-sealing.md` §3.4, DD2-10). The cost is a limit of 64 epochs; the epoch advances only for releases that fix a security flaw in the boot chain. In P2 the bit is set with TPM owner authorization; a fielded CC needs an MSS-signed authorization instead (OI2-05, handed to P3). The same owner authorization, which P2 leaves empty, can also remove the epoch index. Finding "A-01: Anti-rollback epoch can be reset by anyone holding the (empty) TPM owner authorization" in §4.3 covers what that means for anti-rollback.

Salted sessions and response encryption

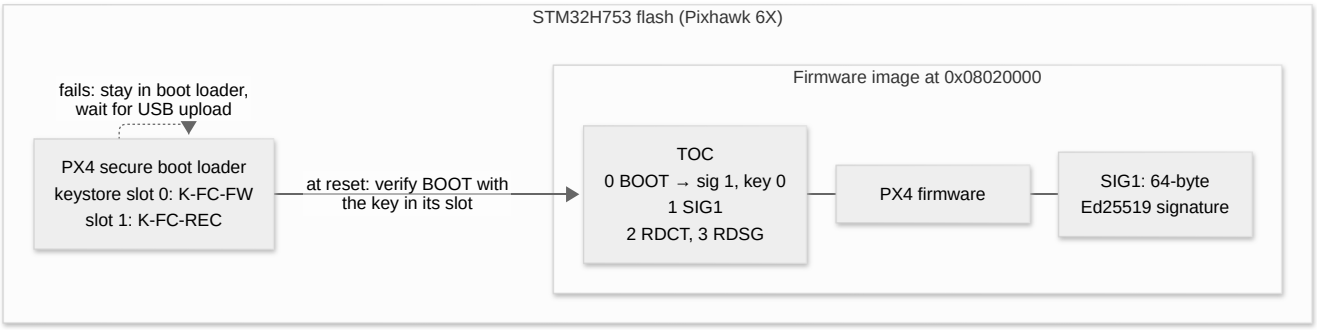
A discrete TPM talks to the SoC over a bus (SPI on the DEBIX). By default, `TPM2_Unseal` returns the secret over that bus in the clear, where a probe on a captured board can read it (adversary tier 2). TPM sessions can encrypt the first parameter of a command or response with a key derived from the session's secret. For that secret to be unknown to a bus observer, the session must be **salted**: the caller encrypts a random salt to a key that already lives in the TPM, and both sides derive the session key from it (P2 `docs/03-measured-boot-sealing.md` §3.6).

How P2 uses it (SR-010.4): `p2-unseal` starts its policy session salted to the parent storage key at `0x81000001`, whose name it checks first, and sets the session to encrypt the response with AES-128 in CFB mode, so the unsealed key crosses the bus only encrypted (P2 `cc/initramfs/p2-unseal.c`:101 and :126–131). This stops passive sniffing. It does not stop an attacker who can reset the TPM on its own and replay the approved measurements (A2-02); P1 records that as THR-034's residual risk, and it is P5's scope. TC-09 checks that the keys never appear in clear; it does not check that the session is salted ("A-04: TC-09 cannot detect an unsalted unseal session, which SR-010.4 forbids" in §4.3).

PX4 Bootloader Secure Boot: the TOC and Ed25519

The PX4 flight controller has its own, much simpler chain. The PX4 boot loader sits at the start of the microcontroller's flash and starts the firmware that follows it. With **Bootloader Secure Boot** (PX4 `v1.18.0-rc1`), it first checks an **Ed25519** signature over the firmware: a 32-byte public key and a 64-byte signature, computed with the monocypher library (P2 `docs/04-fc-secure-boot.md` §4.1, [PX4-SECBOOT], [PX4-BL]).

The firmware image carries a **table of contents (TOC)** at a fixed offset. Each TOC entry names a region of flash, which entry holds its signature, which keystore slot verifies it, and flags. In the `fmu-v6x` build the entries are BOOT (the firmware), SIG1 (the 64-byte signature) and an R&D certificate pair RDCT/RDSG. At reset the boot loader verifies entry 0; if that fails it stays in the boot loader and waits for an upload. Later entries flagged for signature checking are skipped if they do not verify (P2 `docs/04-fc-secure-boot.md` §4.1; `tests/ve08/host/check_image.c`:85–111). The public keys are compiled into the boot loader in up to four slots, `PUBLIC_KEY0` to `PUBLIC_KEY3`; slot 0 is mandatory.



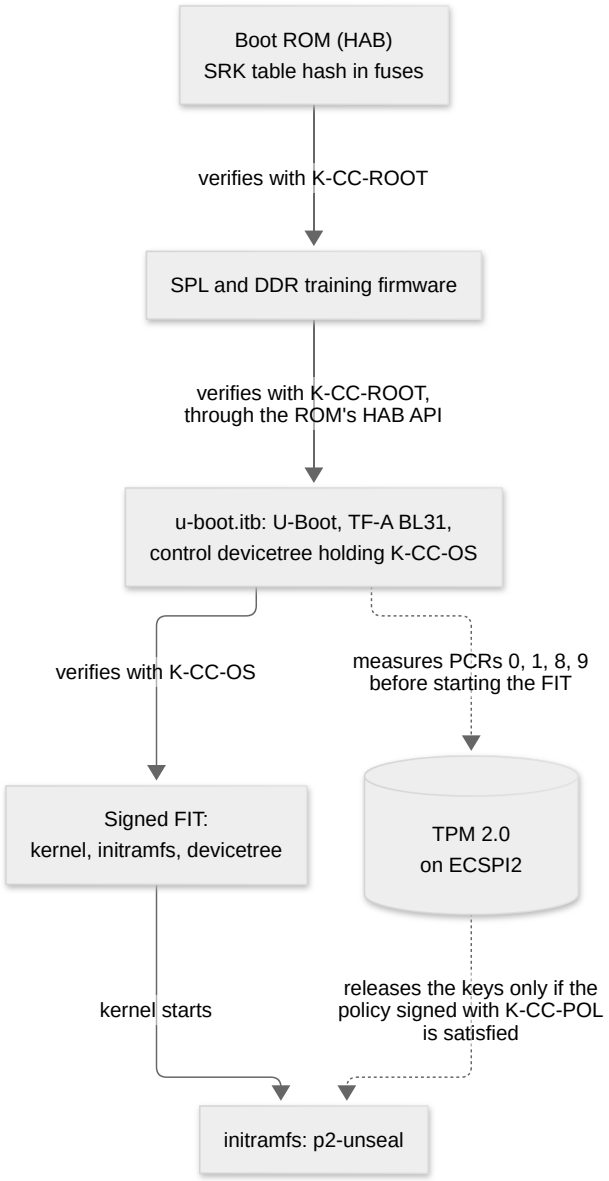
Firmware layout and the check at reset. Addresses and entries are those the TC-11 checker printed for the CI build (`TC-11a.log` in the evidence folder of TC-11).

How P2 uses it (SR-018): K-FC-FW in slot 0 signs the firmware; an offline recovery key K-FC-REC sits in slot 1 (DD2-14). PX4's secure boot has three gaps that P2 documents rather than closes (P2 `docs/04-fc-secure-boot.md` §4.1): it checks only the signature, never a version, so an older signed image still boots (THR-066, closed by SR-045 at the GCS in P3); it sets no flash write-protection, so its integrity rests on the debug port being closed (SR-024, P5); and PX4's signing script reads the raw private key from a JSON file (OI2-04, handed to P3's HSM work).

2.3 The P2 design

The companion computer's boot chain

The CC is a Polyhex DEBIX Model A (NXP i.MX 8M Plus, 2 GB) with an SPI TPM 2.0 on its ECSPI2 controller (DD2-01). It replaced the Raspberry Pi 4 on 2026-10-12, before any hardware was bought: the Pi 4's U-Boot reaches a TPM only through GPIO soft SPI, and its root key is fixed at RSA-2048, which NIST SP 800-57 disallows for new signatures from 2031 (P2 `docs/02-cc-boot-chain.md` §2.1).



The CC boot chain on hardware (after P2 `docs/02-cc-boot-chain.md` §2.2). Solid arrows: the upper stage verifies the lower one with the key named, and refuses to run it if the check fails. Dotted arrows: measurement and key release.

Stage	Verified by	Key	Measured into
SPL and DDR training firmware	Boot ROM (HAB)	K-CC-ROOT (RSA-4096; SRK table hash in fuses)	—
`u-boot.itb`: U-Boot, TF-A BL31, U-Boot control devicetree	SPL, through the ROM's HAB API	K-CC-ROOT	—
Signed FIT: kernel, initramfs, devicetree	U-Boot	K-CC-OS (RSA-3072, SHA-256), required for every configuration	PCR 0 (U-Boot version), PCR 1 (devicetree, command line), PCR 8 (kernel), PCR 9 (initramfs)
Root filesystem	Kernel (dm-verity)	P4 (SR-006)	—

From P2 `docs/02-cc-boot-chain.md` §2.2.

Two verification stages, not one (DD2-02): the fused key table can never be extended, so K-CC-ROOT is kept for the rarely changed boot loader, and K-CC-OS can rotate by shipping a new `u-boot.itb` that carries a new public key.

Locking down U-Boot (SR-005.1)

A signature check protects nothing if something unsigned can change what U-Boot boots. "Boot configuration" in VE-07 therefore means everything that decides what U-Boot boots: its environment, its console and the FIT configuration it picks (P2 `docs/02-cc-boot-chain.md` §2.3). P2's configuration fragment (`cc/uboot/p2.config`, with the board part in `cc/uboot/qemu.config`) sets:

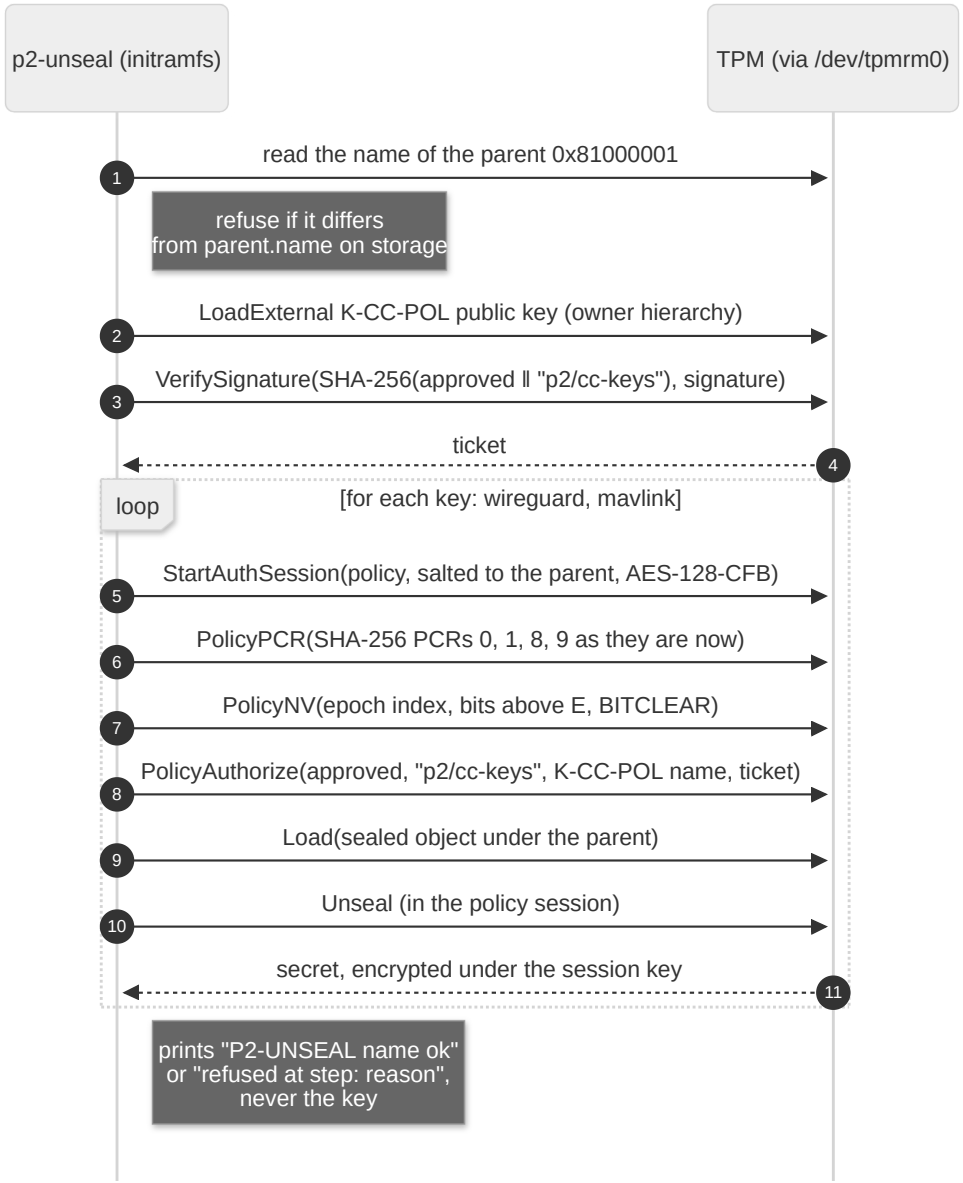
U-Boot setting	Value	Why
`FIT_SIGNATURE`	on; key marked required	Unsigned or wrongly signed configurations are refused
`LEGACY_IMAGE_FORMAT`	off	Legacy uImages carry no signature; `qemu_arm64_defconfig` turns them on
`ENV_IS_NOWHERE`	on	The environment is compiled into the verified image; nothing is read from flash or disk
`BOOTDELAY`	`-2`	Autoboot without checking for a keypress
`BOOTSTD_BOOTCOMMAND`	off	Standard boot would scan disks for unsigned scripts and `extlinux.conf`
`BOOTCOMMAND`	read a fixed-size raw FIT slot, `bootm` it, then `reset`	Any failure resets; nothing reaches a prompt; no filesystem is parsed before the signature check (DD2-05)
`MEASURED_BOOT`	on	Measurements into the TPM

From P2 `docs/02-cc-boot-chain.md` §2.3. `cc/build.sh` fails if any line of the fragments is missing from the built `.config`, and TC-04 checks it again.

On the DEBIX, the board's defconfig keeps the environment in MMC (`ENV_IS_IN_MMC`), so a DEBIX fragment must turn that off and the `.config` check must cover it (P2 `docs/02-cc-boot-chain.md` §2.4). That fragment is not yet written; the CI runs use the QEMU board.

Sealing, unsealing and updates

At provisioning, once per CC, the factory (in P2, the test harness with tpm2-tools) creates the persistent parent, writes the epoch index with no bits set, and seals the two keys under the parent with `authPolicy` = PolicyAuthorize(K-CC-POL, `p2/cc-keys`) (P2 `tests/ve07/provision.py`, docstring). At every boot the initramfs runs `p2-unseal`, a small C program on the TPM software stack's ESAPI (P2 `docs/03-measured-boot-sealing.md` §3.3):



The unseal sequence (P2 `cc/initramfs/p2-unseal.c`). Each refusal case in the tests fails at a different step of it.

A legitimate update (P2 `docs/03-measured-boot-sealing.md` §3.5): the MSS builds the release, predicts its PCRs, builds the approved policy for epoch *E* and signs it; the CC installs the new FIT (and a new boot loader if it changed) with the signed policy; it reboots and unseals with the new policy, the sealed objects untouched; then, if *E* is above the current epoch, it sets bit *E*. If the new chain cannot unseal, the CC does not commit and boots the previous chain; the A/B layout for that belongs to P3 and P4.

Anti-rollback is enforced at key release, not at boot. Neither U-Boot `v2026.07` nor the HAB guides provide image-version checks (P2 `docs/02-cc-boot-chain.md` §2.5). An older signed chain may therefore boot, but it cannot unseal the WireGuard or MAVLink key; without them it cannot reach the GCS, and the FC rejects everything it sends, so the vehicle falls back to the data-link-loss failsafe (P1 DD-05). Refusing to boot an older chain would need a U-Boot patch, considered and not adopted (SR-005.2, DD2-07).

The flight controller

The FC is a Holybro Pixhawk 6X (`px4_fmu-v6x`, STM32H753), the only board with upstream secure-boot build variants at PX4 `v1.18.0-rc1` (DD2-13). The workflow (P2 `docs/04-fc-secure-boot.md` §4.2): generate K-FC-FW and K-FC-REC with PX4's `generate_signing_keys.py`; build `px4_fmu-v6x_bootloader_secureboot` with `PUBLIC_KEY0` = K-FC-FW and `PUBLIC_KEY1` = K-FC-REC and flash it once over SWD; build `px4_fmu-v6x_secureboot` with `BOARD_SECUREBOOT_KEY` = K-FC-FW, which signs the image and wraps it as a `.px4` upload file; upload over USB, then power-cycle to see what the boot loader does at reset.

Recovery (§4.3): a refused image is replaced over USB while the boot loader runs; a lost K-FC-FW is covered by signing with K-FC-REC (which needs a P2 variant of the TOC naming slot 1); losing both keys, or a compromise, needs a new boot loader over SWD. A slot-1 key protects against loss, not compromise: anyone holding either private key can sign firmware that boots.

Keys

Key	Algorithm	Trusted by	Signs	Custody in P2	Custody planned in P3
K-CC-ROOT	Up to four RSA-4096 SRKs	i.MX boot ROM (SRK table hash in fuses)	SPL, `u-boot.itb`	Generated offline for the hardware phase	Offline root tier; CST signs through PKCS#11
K-CC-OS	RSA-3072, SHA-256	U-Boot (control devicetree, required)	FIT configurations	CI: per run. Hardware: local	HSM intermediate tier (`mkimage -N pkcs11`)
K-CC-POL	ECDSA P-256, SHA-256	TPM (its name is in the sealed objects' policy)	Approved policies	CI: per run. Hardware: local	HSM intermediate tier
K-FC-FW	Ed25519	PX4 boot loader, slot 0	PX4 firmware	Local, git-ignored	HSM, once a PKCS#11 Ed25519 signer exists (OI2-04)
K-FC-REC	Ed25519	PX4 boot loader, slot 1	Recovery firmware only	Offline	Offline root tier

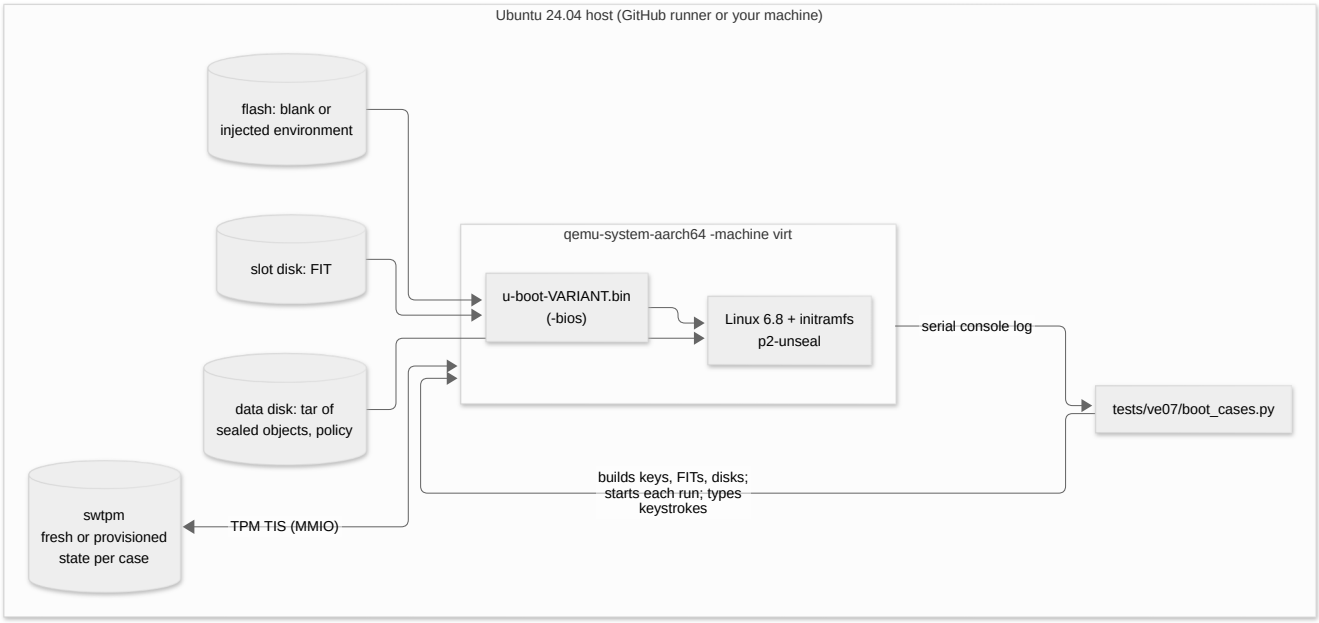
Condensed from P2 `docs/05-key-management.md` §5.1.

Every key in P2 is a development key. CI generates a fresh set on every run and discards it; evidence records only each public key's SHA-256. No private key is committed, and `tools/check_no_secrets.py` fails CI if a PEM, OpenSSH, TSS or PX4 private key, or a WireGuard `PrivateKey =` line, appears in the tree or in an evidence folder (P2 `docs/05-key-management.md` §5.2). No device checks an X.509 chain; each trusts a raw public key or its hash, so P3's hand-off is about custody and ceremony (§5 introduction).

Emulation versus hardware

CI cannot use the DEBIX, so it runs the same U-Boot configuration on QEMU's `virt` machine with swtpm, a software TPM. The differences are deliberate (P2 `docs/02-cc-boot-chain.md` §2.4):

	Hardware (DEBIX Model A)	Emulated (QEMU `virt`)
Root of trust	HAB boot ROM and fused SRK hash; enforced only on a closed chip	None. QEMU loads `u-boot.bin`, trusted by assumption (A2-03)
Where K-CC-OS lives	Control devicetree inside the HAB-signed `u-boot.itb`	Control devicetree passed by QEMU with `-dtb`
TPM	SPI TPM on ECSPI2, driven by U-Boot's `MXC_SPI` (not yet shown, OI2-01)	swtpm through QEMU's `tpm-tis-device` and U-Boot's `TPM2_MMIO`
FIT and CC data	Raw area and a partition on the SD card (layout not yet defined)	Raw virtio disk; CC data as a tar on a second virtio disk
Devicetree	Upstream DEBIX devicetree plus P2's ECSPI2 and TPM nodes	Dumped from QEMU with the same command line used to boot



The emulated harness. Each run boots one U-Boot variant with one FIT, one TPM state and optionally a CC data disk, and the harness matches the console output against patterns.

What a CI run can prove, and what only hardware can (P2 `docs/06-verification.md` §6.2):

- **CI can prove** the U-Boot configuration; that tampered kernels, initramfs, devicetrees and configurations are refused; what U-Boot measures and that it can be predicted; the sealing policy logic (approved state only, signed approval, epoch anti-rollback, objects bound to one TPM, encrypted unseal); the FC signing pipeline and image format; and that each test can detect what it looks for, through positive and control runs.
- **Only hardware can prove** a hardware root of trust refusing a modified boot loader on a closed chip (TC-02); U-Boot reaching a real TPM over ECSPI (OI2-01); sealing on a real TPM rather than swtpm (A2-04), including removal of real storage; and the PX4 boot loader enforcing signatures at reset on the STM32H753, with its recovery paths (TC-12 to TC-14).
- **Neither** proves resistance to physical attack: TPM bus interposers, TPM reset (A2-02), debug-port access or fault injection. Those belong to P5.

2.4 Test cases

P2 has 14 test cases: 10 run in CI (emulated, or on the host for the FC) and 12 on hardware. CI only: TC-09, TC-11. Hardware only: TC-02, TC-12, TC-13, TC-14. Passed in CI: TC-01, TC-03, TC-04, TC-05, TC-06, TC-07, TC-08, TC-09, TC-10, TC-11. Passed on hardware: 0 of 12.

ID	Event	Test case	Verifies	CI	Hardware
TC-01	VE-07	Signed chain boots	SR-005, SR-005.1	passed	planned
TC-02	VE-07	Modified boot loader refused by the hardware root of trust	SR-005	–	planned
TC-03	VE-07	Modified kernel, initramfs or devicetree refused	SR-005	passed	planned
TC-04	VE-07	Modified boot configuration refused	SR-005, SR-005.1	passed	planned
TC-05	VE-07	Keys unseal in the measured, unmodified boot state	SR-010, SR-010.1, SR-010.2	passed	planned
TC-06	VE-07	Keys do not unseal from a modified boot state	SR-010, SR-010.1	passed	planned
TC-07	VE-07	Keys do not unseal from removed storage	SR-010	passed	planned
TC-08	VE-07	Rollback and unapproved policies refused	SR-010.2, SR-010.3	passed	planned
TC-09	VE-07	Unseal traffic is encrypted	SR-010.4	passed	–

ID	Event	Test case	Verifies	CI	Hardware
TC-10	VE-07	Predicted PCRs match the measured PCRs and the event log	SR-010.1	passed	planned
TC-11	VE-08	FC images signed and checked on the host	SR-018	passed	–
TC-12	VE-08	Signed firmware starts PX4	SR-018	–	planned
TC-13	VE-08	Unsigned, modified and wrong-key firmware stay in the boot loader	SR-018	–	planned
TC-14	VE-08	Recovery paths work	SR-018	–	planned

Generated from the P2 snapshot's `data/testcases.csv`. "–" means the case does not run there.

Two rules apply to every case (P2 `docs/06-verification.md` §6.4). **Controls first:** a negative case counts only if its positive control passed in the same run (TC-01 for every case; TC-05 as well for the sealing refusals), and where an attack would succeed against a weaker setup, a control run shows it succeeding there, so the test can be seen to detect it. **Provenance:** only the unchanged artifact of a GitHub Actions run counts as CI evidence; a run anywhere else is marked `local`.

For each case below, the card is generated from `data/testcases.csv`; the text after it says what a pass shows, what it does not, and what was emulated.

TC-01: the signed chain boots

TC-01	Signed chain boots
Event	VE-07 CC verified boot and key sealing
Verifies	SR-005, SR-005.1
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	Boot the chain signed with the project keys. Emulated: QEMU virt, U-Boot and swtpm (tests/ve07). Hardware: DEBIX Model A booting a HAB-signed flash.bin, with the SRK hash fused.
Pass criteria	U-Boot verifies the FIT configuration signature with the required key and the kernel reaches the initramfs. On hardware, HAB also reports no events.

Emulated: QEMU `virt` with the hardened U-Boot, a FIT signed with a per-run K-CC-OS development key, and a blank swtpm. The run passes when the console shows `sha256,rsa3072:cc-os-dev+ OK` (the configuration signature verified with the required key), `Starting kernel` and the initramfs marker `P2-INITRAMFS-OK`, and shows no injected-environment marker and no console prompt (P2 `tests/ve07/boot_cases.py`:724–725).

A pass shows that the hardened U-Boot verifies the required signature and boots the signed chain to user space. It is the positive control for every other VE-07 case. **It does not show** the hardware root of trust: QEMU loads `u-boot.bin` and supplies the control devicetree holding K-CC-OS (A2-03). The hardware run adds the HAB check: no HAB events on the fused board.

TC-02: a modified boot loader is refused by the hardware root of trust

TC-02	Modified boot loader refused by the hardware root of trust
Event	VE-07 CC verified boot and key sealing
Verifies	SR-005
CI	does not run there
Hardware	planned
Procedure	Hardware only, with the SRK hash fused. Boot a flash.bin whose SPL has one byte changed, then one whose u-boot.itb is signed by a key outside the fused SRK table. Run once on the open chip, then again after the chip is closed.
Pass criteria	On the closed chip, both images are refused and U-Boot never runs. The open-chip run is a check only: HAB logs a failure event for each image, and none for the correct one, but boots them anyway.

Hardware only. Emulation has no root of trust to test (A2-03). The run needs the SRK hash fused and, for the real test, the chip closed: two permanent steps, each taken only with the owner's go-ahead (OI2-02). The open-chip run is a check that HAB *notices* the bad images; only the closed-chip run shows them refused.

A pass would show that the boot ROM refuses an SPL with one byte changed and a `u-boot.itb` signed by a key outside the fused SRK table, so U-Boot never runs. **It would not show** resistance to fault injection or debug-port attacks on the ROM's check, which P2 leaves to P5 (P2 `docs/01-scope-and-trace.md` §1.5), nor anything about images other than the two in its procedure. Until it passes, the start of the CC's chain of trust is design, not evidence.

TC-03: a modified kernel, initramfs or devicetree is refused

TC-03	Modified kernel, initramfs or devicetree refused
Event	VE-07 CC verified boot and key sealing
Verifies	SR-005
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	After signing, change one byte in the kernel, the initramfs and the devicetree payload of the FIT, in turn.
Pass criteria	U-Boot reports a hash mismatch for the changed image, does not start the kernel and resets without offering a console.

Emulated: 3 test runs, TC-03a to TC-03c. Each inverts one byte in the middle of one image's data in the signed FIT, after signing, as an attacker with storage access would. The configuration signature still verifies, because the signed hash values are untouched; the per-image hash check then fails with `Bad hash value for 'hash' hash node in 'kernel' image node` (or `ramdisk`, `fdt`), and U-Boot prints `resetting ...` without starting the kernel or offering a prompt (P2 `tests/ve07/boot_cases.py`:726–735).

A pass shows that U-Boot checks every image's data against the signed hashes, and resets on failure. **It does not show** this on the DEBIX's SD card or with the DEBIX build of U-Boot.

TC-04: a modified boot configuration is refused

TC-04	Modified boot configuration refused
Event	VE-07 CC verified boot and key sealing
Verifies	SR-005, SR-005.1
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	In turn: point the FIT default at an added unsigned configuration; boot an unsigned FIT; boot a FIT signed by a rogue key with the same key name; boot a legacy uImage; inject a boot-loader environment with another bootcmd into flash; type at the console during a good boot and after a refused boot. Control runs repeat the environment and legacy-image cases on stock qemu_arm64_defconfig, and the console cases on the P2 build with only the autoboot delay restored or only the final reset removed.
Pass criteria	Every attempt is refused or ignored and no console prompt appears. The control runs show the stock configuration accepts the injected environment and the legacy image, and the same keystrokes reach a prompt when either console setting is relaxed.

Emulated: the widest case, with 8 test runs and 4 control runs (P2 `tests/ve07/boot_cases.py`:736–767):

- **TC-04a** checks that the built `.config` holds every P2 setting, that each control or test build differs from it in exactly its one setting, and that the control devicetree marks the key required.
- **TC-04b** adds an unsigned configuration after signing and makes it the default: `No 'signature' subnode found`, `Failed to verify required signature`.
- **TC-04c** boots a FIT with no signature at all; **TC-04d** one signed by a rogue key that carries the same key name (`Verification failed for 'conf' config node`).

- **TC-04e** puts a legacy uImage in the boot slot (`Wrong Image Type for bootm command`). Its control boots the same uImage on stock U-Boot, through an injected environment.
- **TC-04f** injects a boot-loader environment with another boot command into flash; the hardened U-Boot ignores it and boots the signed chain. Its control shows stock U-Boot running the injected environment.
- **TC-04g** types at the serial console from the moment it comes up until the kernel starts; no prompt appears. Its control, the hardened build with only the autoboot delay restored, stops at a prompt with the same keystrokes.
- **TC-04h** types after a refused boot; the board resets and refuses again, five boots in all, and never shows a prompt. Its control, the hardened build without the final `reset`, falls through to a prompt and answers `help`.

A pass shows that in this U-Boot build none of these routes changes what boots, and the controls show that each route works on a weaker configuration, so the test can detect it. **It does not show** the DEBIX configuration, whose defconfig keeps the environment in MMC and needs its own fragment (P2 `docs/02-cc-boot-chain.md` §2.4), nor console attacks beyond serial keystrokes. TC-04a compares the built `.config` with the same fragments it was built from, so it cannot notice a hardening line deleted from a fragment; only the boot cases can (“A-06: TC-04a's .config check is circular; the DEBIX environment case targets the wrong location” in §4.3).

TC-05: the keys unseal in the measured, unmodified boot state

TC-05	Keys unseal in the measured, unmodified boot state
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010, SR-010.1, SR-010.2
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	Boot the signed chain with a signed policy for its predicted PCR values and unseal the WireGuard and MAVLink signing keys in the initramfs.
Pass criteria	Both keys unseal and match the provisioned values.

Emulated: before anything boots, the harness provisions an swtpm with tpm2-tools as a factory would: the persistent parent, the epoch index and the two keys, generated per run and sealed; it keeps only their SHA-256 values. It then approves the release as the MSS would, with a per-run K-CC-POL development key and `tools/approve_policy.py`, and packs the CC data as a tar on a second virtio disk (P2 `tests/ve07/README.md`). The run passes when the initramfs prints `P2-UNSEAL wireguard ok` and `P2-UNSEAL mavlink ok`, the SHA-256 of each unsealed key equals that of the key sealed, the provisioned objects have the specified attributes and names, and the PCR checks of TC-10 also hold.

A pass shows the whole sealing design working end to end on swtpm with U-Boot's real measurements: the signed approval, the epoch check, the encrypted session and the unseal. It is the positive control for TC-06 to TC-09. **It does not show** a real TPM (A2-04), U-Boot measuring into an SPI TPM (OI2-01), or a real provisioning ceremony; the keys are random test keys, not keys in use. Nor does it show that the session is salted: it also passes with the salt removed (“A-04: TC-09 cannot detect an unsalted unseal session, which SR-010.4 forbids” in §4.3).

TC-06: the keys do not unseal from a modified boot state

TC-06	Keys do not unseal from a modified boot state
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010, SR-010.1
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	Boot chains that each differ in one measured item (boot-loader version, devicetree, kernel command line, kernel, initramfs), all signed with the project key but with no approved policy for that state.
Pass criteria	Unseal fails in every case.

Emulated: 5 test runs, each a chain that differs from the approved release in one measured item but is built and signed with K-CC-OS like a release, so it boots: another U-Boot version string (PCR 0), one more kernel argument (PCR 1), one more devicetree property (PCR 1), the same kernel with other bytes in its gzip header (PCR 8), and one more file in the initramfs (PCR 9). The harness checks that only the intended PCR differs from the approved release's prediction. Each run must boot (`P2-INITRAMFS-OK`) and show both keys `refused at PolicyAuthorize` (P2 `tests/ve07/boot_cases.py`:771–791).

Why at `PolicyAuthorize` and not at `PolicyPCR: p2-unseal` passes an empty PCR digest, so `PolicyPCR` folds in whatever the PCRs hold now (P2 `cc/initramfs/p2-unseal.c`:106). The mismatch surfaces when `PolicyAuthorize` finds that the session digest is not the approved policy.

A pass shows that a change to any measured item denies the keys even to a chain signed with the project key, which is what makes key release independent of signature checking. **It does not show** anything about items U-Boot does not measure, such as the root filesystem or anything after the initramfs (PCRs 10 and up; P4). Nor does it show that the keys stop being releasable once the initramfs has run: no PCR is extended after the unseal, so later code in an approved boot can unseal them again (“A-02: PCRs are never capped after unseal; keys stay releasable for the rest of an approved boot” in §4.3). And its kernel variant changes only the gzip header, so on its own it cannot show that U-Boot measures the whole kernel (“A-03: TC-06's kernel variant changes only the gzip header; a shared measurement fault passes every case” in §4.3).

TC-07: the keys do not unseal from removed storage

TC-07	Keys do not unseal from removed storage
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	Copy the sealed objects and signed policies from CC storage and try to load and unseal them on a second TPM. Emulated: a second swtpm instance. Hardware: a second TPM module.
Pass criteria	The sealed objects do not load on the second TPM.

Emulated: 2 test runs. A second swtpm provisioned the same way stands in for another CC's TPM. TC-07a boots this CC's data on it: `p2-unseal` refuses at `parent name`, because the second TPM's parent is not the one recorded. TC-07b changes the recorded parent name to the second TPM's: the TPM itself then refuses at `Load` with `integrity check failed`, because only the original parent can unwrap the sealed objects (P2 `tests/ve07/boot_cases.py`:793–797).

A pass shows that the sealed objects on storage are useless without their own TPM. **It does not show** an attack on that TPM itself (P5), and the hardware run repeats it with a second TPM module. TC-07a's refusal at `parent name` relies on a file kept on the same unprotected storage (“A-25: The parent-name check trusts a file on unprotected storage; its purpose is untested” in §4.3).

TC-08: rollback and unapproved policies are refused

TC-08	Rollback and unapproved policies refused
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010.2, SR-010.3
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	After an update to epoch N+1 is committed, boot the older signed chain with its signed epoch-N policy; then present a policy signed by a rogue key.
Pass criteria	Unseal fails in both cases and the current chain still unseals.

Emulated: 4 test runs. The harness builds a release 2 (one more file in the initramfs), has the MSS approve it at epoch 1, and commits epoch 1 on a copy of the provisioned TPM (P2 `tests/ve07/boot_cases.py`:798–811):

- **TC-08a**, release 1 with its epoch-0 policy after the commit: refused at `PolicyNV` (`policy failure in math operation`).
- **TC-08b**, release 2 with its epoch-1 policy: still unseals. This is the check that the commit did not lock out the current release.
- **TC-08c**, a policy signed by a rogue key, with the rogue public key also placed on storage: the TPM verifies the rogue signature, but `PolicyAuthorize` then names the rogue key, so the session digest cannot match the sealed objects' `authPolicy`: refused at `Unseal` (`a policy check failed`).
- **TC-08d**, a rogue-signed policy with the real K-CC-POL public key on storage: refused at `VerifySignature` (`the signature is not valid`).

A pass shows anti-rollback at key release against an older chain that does not touch the TPM, and that only K-CC-POL can approve a state. **It does not show** how a fielded CC would commit an epoch without a long-lived owner secret on the CC (OI2-05, P3). Nor does it show anti-rollback against a caller with TPM owner authorization, who can remove and re-create the epoch index (“A-01: Anti-rollback epoch can be reset by anyone holding the (empty) TPM owner authorization” in §4.3), or that a K-CC-POL signature made for another policyRef is refused (“A-05: The policyRef binding is untested; emptying it leaves every case green” in §4.3).

TC-09: unseal traffic is encrypted

TC-09	Unseal traffic is encrypted
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010.4
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	does not run there
Procedure	Record TPM command and response traffic during unseal from the swtpm I/O log. Control run: unseal over a session without encryption.
Pass criteria	The unsealed key bytes never appear in the recorded traffic; in the control run they do.

Emulated, CI only: 1 test run and 1 control run. swtpm logs every TPM command and response at log level 20; the harness parses the log, checks that it saw one `TPM2_Unseal` per key, each answered with success, and searches every frame for each key's bytes. The control run boots an initramfs with `p2-unseal` built with response encryption off (`-DP2_CONTROL_CLEAR_RESPONSE`), under its own approved policy; there, both keys appear in the `Unseal` responses. The I/O logs kept as evidence have any key bytes replaced by `--`, with the position and SHA-256 of each replacement recorded (P2 `tests/ve07/boot_cases.py`:250–268, :813–820).

A pass shows that with SR-010.4's session settings the keys never cross the TPM interface in the clear. P2 plans no hardware run of this case (its `hw_status` in `data/testcases.csv` is `n/a`) and notes that on hardware the same bytes cross SPI (P2 `tests/ve07/README.md`); that is P2's statement, not a measurement (“A-10: SR-010.4 is verified only on swtpm's log, with no hardware run, contrary to A2-04” in §4.3). **It does not show** that the session is salted, which SR-010.4 requires: the case still passes with the salt removed (“A-04: TC-09 cannot detect an unsalted unseal session, which SR-010.4 forbids” in §4.3). Nor does it show resistance to active bus attacks or a TPM reset (A2-02).

TC-10: predicted PCRs match the measured PCRs and the event log

TC-10	Predicted PCRs match the measured PCRs and the event log
Event	VE-07 CC verified boot and key sealing
Verifies	SR-010.1
CI	passed (evidence: `evidence/ve07-ci/20261013-d9149f6/`)
Hardware	planned
Procedure	Predict PCRs 0, 1, 8 and 9 from the release artifacts with tools/predict_pcrs.py, boot, read the PCRs and replay the event log.
Pass criteria	The predicted, measured and replayed values are equal.

Emulated: `tools/predict_pcrs.py` computes PCRs 0, 1, 8 and 9 from the signed FIT and the hardened `u-boot.bin` before anything boots. The initramfs prints the PCRs from sysfs and the event log Linux received; the harness checks that every policy PCR moved from its reset value, that the log's events are the predicted ones, and that the predicted, measured and replayed values are equal (P2 `tests/ve07/boot_cases.py`:821–822, :865–879).

A pass shows that the MSS can compute the values a release will produce, which is what makes signed approvals possible (TC-05). **It does not show** measurement into a real TPM over SPI (OI2-01). And because it compares U-Boot with a predictor written from the same reading of U-Boot, a mistake both share passes (“A-03: TC-06's kernel variant changes only the gzip header; a shared measurement fault passes every case” in §4.3).

TC-11: FC images signed and checked on the host

TC-11	FC images signed and checked on the host
Event	VE-08 FC boots only signed firmware
Verifies	SR-018
CI	passed (evidence: `evidence/ve08-ci/20261013-aac4299/`)
Hardware	does not run there
Procedure	Build px4_fmu-v6x_bootloader_secureboot and px4_fmu-v6x_secureboot at the pinned PX4 tag with a per-run key. Check signed, unsigned, modified-signed and wrong-key images with PX4's own verification code compiled for the host.
Pass criteria	Only the image signed with the run key verifies.

On the host, CI only: PX4 is cloned at the pinned commit and built in PX4's own build container, pinned by digest, with a per-run K-FC-FW key: the boot loader with the key in slot 0, and the firmware signed with it. The firmware is also signed with PX4's committed test key, which is public and so a safe stand-in for "a key the boot loader does not trust". The checker is PX4's own `find_toc()`, `verify_app()`, stub keystore and monocypher, compiled for a 32-bit host so the TOC's addresses keep their 4 bytes, with the image mapped at its flash address `0x08020000` (P2 `tests/ve08/README.md`). Runs: TC-11a signed (starts); TC-11b unsigned, TC-11c one byte changed, TC-11d test-key signed (all stay in the boot loader); TC-11d-control, the same test-key image with the test key in slot 0 (starts), so TC-11d fails on the key alone. Static checks: the boot loader holds the run key once and not the test key, and the signed image is the built image padded to 4 bytes plus a 64-byte signature.

A pass shows that the pinned PX4 builds a boot loader trusting only the run key, signs the firmware the way the boot loader checks it, and that PX4's verification code accepts only that image. **It does not show** the check at reset on the STM32H753, the upload path or recovery (TC-12 to TC-14). Nor does it show PX4's exact decision. The checker re-implements the boot loader's loop and the Ed25519 glue, which already differ from PX4 in one check, and it fixes the flash size at 2 MiB (“A-11: TC-11 re-implements PX4's boot decision and fixes flash at 2 MiB; the copies already diverge” in §4.3).

TC-12 to TC-14: the flight controller on hardware

Hardware only. Hardware status: TC-12 planned, TC-13 planned, TC-14 planned. These need a Pixhawk 6X, an SWD probe to flash the boot loader, and power cycles: the power cycle shows what the boot loader does at reset, not only what the uploader reports (P2 `docs/04-fc-secure-boot.md` §4.2).

TC-12	Signed firmware starts PX4
Event	VE-08 FC boots only signed firmware
Verifies	SR-018
CI	does not run there
Hardware	planned
Procedure	Flash the secure boot loader over SWD, upload firmware signed with the project key over USB, then power-cycle.
Pass criteria	The board leaves the boot loader and PX4 starts, shown by the console log and USB enumeration.

A pass would show that the secure boot loader, built with the project key and flashed over SWD, starts firmware signed with that key on the STM32H753 after a power cycle. It is the positive control for TC-13. **It would not show** that older signed firmware is stopped: it is not, because the boot loader checks no version (THR-066, mitigated at the GCS by SR-045 in P3).

TC-13	Unsigned, modified and wrong-key firmware stay in the boot loader
Event	VE-08 FC boots only signed firmware
Verifies	SR-018
CI	does not run there
Hardware	planned
Procedure	Upload in turn an unsigned image, a signed image with one byte changed, and an image signed with PX4's public test key; power-cycle after each.
Pass criteria	The uploader reports a signature failure and, after each power cycle, the board stays in the boot loader.

A pass would show that the boot loader enforces the check at reset on the real chip, for the three kinds of image TC-11 checked on the host; the wrong-key image is signed with PX4's public test key. **It would not show** resistance to an attacker with SWD access, who can replace the boot loader itself (SR-024, P5), or to fault injection during the check.

TC-14	Recovery paths work
Event	VE-08 FC boots only signed firmware
Verifies	SR-018
CI	does not run there
Hardware	planned
Procedure	After a refused image, upload the signed image over USB. Then change the boot loader's trusted key over SWD, and finally restore the stock boot loader over SWD.
Pass criteria	Each step returns the board to the intended state.

A pass would show that a board left in the boot loader by a refused image can be recovered over USB, that the trusted key can be changed over SWD, and that the stock boot loader can be restored, so a hardware run does not leave the board unusable. **It would not show** recovery by signing with K-FC-REC in slot 1, which is not in its procedure and needs a P2 variant of PX4's TOC (P2 `docs/04-fc-secure-boot.md` §4.3). And as written, its pass criterion ("the intended state") does not say what a pass is (“A-24: DD2-14's recovery key is neither built nor tested; TC-14 cannot fail as written” in §4.3).

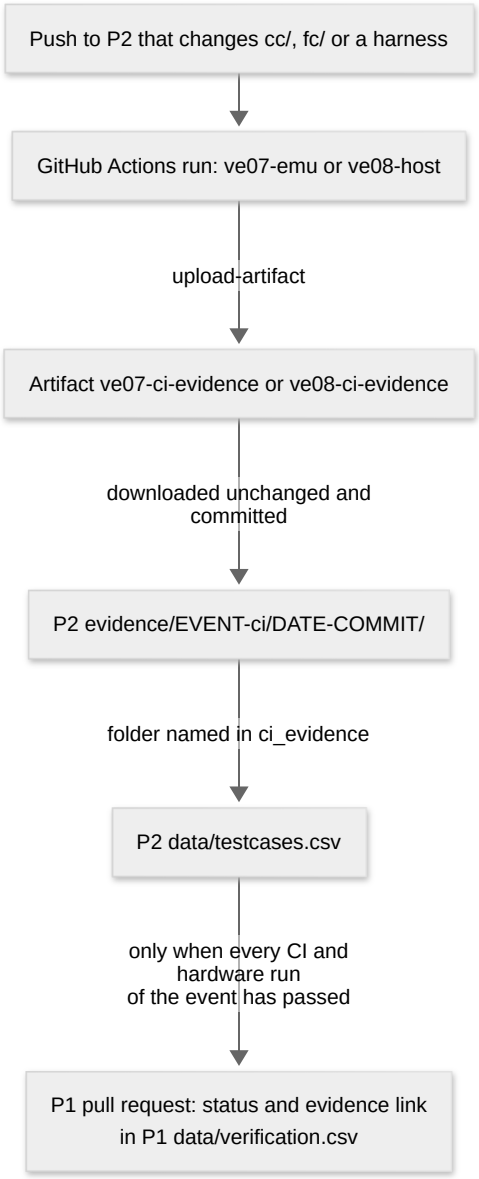
2.5 Reading the evidence

Where it lives

Evidence lives in P2 under `evidence/<event>-<ci|hw>/<YYYYMMDD>-<commit>/`, where the commit is the code commit the run tested (P2 `docs/06-verification.md` §6.4; `evidence/README.md`). A `ci` folder is the unchanged artifact of a GitHub Actions run, downloaded and committed. Each folder holds:

- `summary.md`, the readable report;
- `summary.json`, with every check behind each result, the run's provenance (`environment`: run ID, workflow, commit, runner image), tool versions, the host packages compared with their pins, the SHA-256 of every built artifact and public key, and each case's result;
- one log per run: the serial console for VE-07, the image checker's output for VE-08; and for TC-09 the redacted swtpm I/O logs.

A test case's evidence column in `data/testcases.csv` points at the newest run that covers it; older folders stay as the record of what passed at their commit.



From a CI run to P1. P1's status is planned for VE-07 and planned for VE-08; P2's roll-up is in progress for VE-07 and in progress for VE-08.

The current CI evidence

VE-07's test cases cite `evidence/ve07-ci/20261013-d9149f6/`: GitHub Actions run 38349138261 of workflow `ve07-emu` on runner image `ubuntu24 20261004.327.1`, testing code commit `d9149f6eb9305dc1e477319f2e5c03f517ce12c3`, dated 2026-10-13T05:05:52Z; 31 of 31 runs passed, 5 of them control runs. Its host recorded:

Tool	Version the run recorded
os	Linux-6.17.0-1022-azure-x86_64-with-glibc2.39
python	3.12.3
qemu	QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
swtpm	TPM emulator version 0.7.3, Copyright (c) 2014-2021 IBM Corp.
tpm2-tools	tool="tpm2_createprimary" version="5.6" tctis="libtss2-tctildr" tcti-default=tcti-device
openssl	OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
dtc	Version: DTC 1.7.0

Tool	Version the run recorded
mkimage	mkimage version 2026.07

Run	Role	What it boots or checks	Result
TC-04a	test	Built .config holds every P2 setting; each control build changes exactly one	pass
TC-01	positive	Signed chain boots to the initramfs	pass
TC-03a	test	Kernel changed after signing	pass
TC-03b	test	Initramfs changed after signing	pass
TC-03c	test	Devicetree changed after signing	pass
TC-04b	test	Unsigned configuration added after signing and made default	pass
TC-04c	test	Unsigned FIT	pass
TC-04d	test	FIT signed by a rogue key with the same key name	pass
TC-04e	test	Legacy uImage in the boot slot	pass
TC-04e-control	control for TC-04e	Stock U-Boot boots the same legacy uImage (via injected environment)	pass
TC-04f	test	Environment injected into flash is ignored	pass
TC-04f-control	control for TC-04f	Stock U-Boot runs the injected environment	pass
TC-04g	test	Keystrokes from console start to the end of a good boot reach no prompt	pass
TC-04g-control	control for TC-04g	The same keystrokes stop autoboot when only the delay is restored	pass
TC-04h	test	Keystrokes after a refused boot reach no prompt; the board resets and refuses again	pass
TC-04h-control	control for TC-04h	The same keystrokes reach a prompt when only the reset is removed	pass
TC-05	test	Keys unseal in the measured, unmodified boot state	pass
TC-06a	test	Boot loader with another version string (PCR 0)	pass
TC-06b	test	Kernel command line with one more argument (PCR 1)	pass
TC-06c	test	Devicetree with one more property, re-signed (PCR 1)	pass
TC-06d	test	Kernel image with other bytes, re-signed (PCR 8)	pass
TC-06e	test	Initramfs with one more file, re-signed (PCR 9)	pass
TC-07a	test	This CC's data on a second CC's TPM	pass
TC-07b	test	The same, with the parent name changed to the second TPM's	pass
TC-08a	test	Release 1 with its epoch-0 policy, after epoch 1 is committed	pass
TC-08b	test	Release 2 with its epoch-1 policy, after the commit, still unseals	pass
TC-08c	test	Release 2 with a policy signed by a rogue key, rogue public key on storage	pass
TC-08d	test	Release 2 with a policy signed by a rogue key, K-CC-POL public key on storage	pass
TC-09	test	Neither key appears in the TPM traffic of a TC-05 boot	pass
TC-09-control	control for TC-09	Both keys appear in it when the agent leaves the response in the clear	pass
TC-10	test	Predicted PCRs equal the measured PCRs and the event-log replay	pass

Every run of the VE-07 evidence, generated from its `summary.json`.

What `p2-unseal` printed in each sealing run:

Run	U-Boot	FIT	TPM	CC data	What p2-unseal printed
TC-05	hardened	good	provisioned	cc	ok
TC-06a	tc06-version	good	provisioned	cc	refused at PolicyAuthorize: tpm:parameter(1):value is out of range or is not correct for the context
TC-06b	tc06-bootargs	good	provisioned	cc	refused at PolicyAuthorize: tpm:parameter(1):value is out of range or is not correct for the context
TC-06c	hardened	tc06-devicetree	provisioned	cc	refused at PolicyAuthorize: tpm:parameter(1):value is out of range or is not correct for the context

Run	U-Boot	FIT	TPM	CC data	What p2-unseal printed
TC-06d	hardened	tc06-kernel	provisioned	cc	refused at PolicyAuthorize: tpm:parameter(1):value is out of range or is not correct for the context
TC-06e	hardened	tc06-initramfs	provisioned	cc	refused at PolicyAuthorize: tpm:parameter(1):value is out of range or is not correct for the context
TC-07a	hardened	good	second	cc	refused at parent name: missing or wrong CC data
TC-07b	hardened	good	second	cc-moved	refused at Load: tpm:parameter(1):integrity check failed
TC-08a	hardened	good	committed	cc	refused at PolicyNV: tpm:error(2.0): policy failure in math operation or an invalid authPolicy value
TC-08b	hardened	r2	committed	cc-r2	ok
TC-08c	hardened	r2	committed	cc-r2-rogue-key	refused at Unseal: tpm:session(1):a policy check failed
TC-08d	hardened	r2	committed	cc-r2-rogue-sig	refused at VerifySignature: tpm:parameter(2):the signature is not valid
TC-09	hardened	good	provisioned	cc	ok
TC-09-control	hardened	ctl-clear	provisioned	cc-ctl-clear	ok (control build: response in the clear)

VE-08's test case cites `evidence/ve08-ci/20261013-aac4299/`: GitHub Actions run 38351585198 of workflow `ve08-host`, code commit `aac42999041002065a08c0735633be4155403467`, PX4 `v1.18.0-rc1` at `fca3df865af36124a28c9d607e850f111dbaaea9`, built with `arm-none-eabi-gcc (15:13.2.rel1-2) 13.2.1 20231009`; 5 of 5 runs passed, 1 of them a control run.

Run	Role	What it boots or checks	Result
TC-11a	positive	Image signed by the build with the run key	pass
TC-11b	test	Image as built, unsigned	pass
TC-11c	test	Signed image with one byte changed after signing	pass
TC-11d	test	Image signed with PX4's public test key	pass
TC-11d-control	control for TC-11d	The same image starts when the test key is in slot 0	pass

Static check	Result
the boot loader holds the run key once	pass
the boot loader does not hold PX4's public test key	pass
the signed image is the built image, padded to 4 bytes, plus a 64-byte signature	pass
the .px4 envelope carries the signed image and marks it signed	pass

How to check a claim yourself

Take TC-05 as the example. In its folder, open `summary.md`, find the TC-05 row, then open `TC-05.log`. The console log is raw, with terminal escape codes; the harness removes them before matching (P2 `tests/ve07/boot_cases.py`:169–171). The lines that matter, from the CI log at `db31c14` with the escape codes removed (`...` on a line of its own marks lines left out):

```
## Loading kernel (any) from FIT Image at 50000000 ...
   Using 'conf' configuration
   Verifying Hash Integrity ... sha256,rsa3072:cc-os-dev+ OK
...
Starting kernel ...
...
P2-INITRAMFS-OK kernel=6.8.0-146-generic
P2-PCR sha256 0 30C8A64625E906BD4CBB92442CBB825D80A4E91559AECE4C5254E554A7FB54E0
...
P2-UNSEAL wireguard ok
P2-UNSEAL mavlink ok
P2-KEY wireguard sha256 fe0b3314b248899f8742a80fda8e684677321f8f806728a565db83f6118ee713
P2-KEY mavlink sha256 462f8c1f155618730c266a891d397bbfd86858aeb382b01f2b517f599d075e04
```

Then compare: the `P2-KEY` digests must equal the "SHA-256 sealed" values in `summary.md` (they are the per-run keys' digests; the keys themselves are never written); the `P2-PCR` values must equal the TC-10 table. A refusal looks like this, from `TC-08a.log`:

```
P2-UNSEAL wireguard refused at PolicyNV: tpm:error(2.0): policy failure in math operation or an invalid
authPolicy value
```

and a refused boot like this, from `TC-03a.log`:

```
   Verifying Hash Integrity ... sha256,rsa3072:cc-os-dev+ OK
   Trying 'kernel' kernel subimage
...
     Hash value:   6001f54164a5201cd5a80e0a09bb05c43fa6e910a0abfe0f5a4e8d7d781cc737
   Verifying Hash Integrity ... sha256 error!
Bad hash value for 'hash' hash node in 'kernel' image node
Bad Data Hash
ERROR -2: can't get kernel image!
resetting ...
```

The `Hash value` is the signed hash of the kernel as built (the `Image.gz` digest in step 6 of §2.7); the data no longer matches it, because one byte was inverted after signing.

For VE-08, each log is a few lines from the checker; `TC-11a.log` ends `P2-FC result: starts the application at 0x08020000` and `TC-11d.log` ends `P2-FC result: stays in the boot loader: TOC entry 0 does not verify`.

Three things to check before trusting a folder, and a fourth below: `summary.md` says "Where: GitHub Actions run …" rather than "local run (not CI evidence)"; "Host packages match the pins" says yes; and the code commit is one whose `cc/` or `fc/` matches what you are evaluating. The run URL in `summary.json` leads to the run on GitHub, where its uploaded artifact can be compared with the committed folder for as long as GitHub retains the artifact. The audit compared all six on 2026-10-13 and found each byte-identical. The artifacts expire in January 2027, and P2 records no artifact digest that would allow the check after that (“A-08: Evidence provenance expires with the GitHub artifacts (2027-01-09 to 01-11); no digests recorded” in §4.3). A fourth check is needed because P2's validator accepts any existing file as the evidence for a pass: open the cited folder's `summary.json` and confirm it ran the test case and passed it (“A-07: The validator accepts any existing file as evidence for a pass” in §4.3).

2.6 Open items

P2's own open items, as its documents state them; none is closed by a test case yet:

ID	Item	Closed by
OI2-01	U-Boot driving the SPI TPM on ECSPI2 of the DEBIX has not been shown (A2-06). Fallback: measure from the verified initramfs, recorded as a deviation from SR-010.1	First hardware spike
OI2-02	HAB enforces only on a closed chip; burning the SRK hash and closing the chip are separate permanent steps	TC-02, each step with the owner's go-ahead
OI2-04	PX4's signer needs the raw Ed25519 private key; SR-034 needs signing inside an HSM	P3
OI2-05	Advancing the epoch uses TPM owner authorization; a fielded CC needs an MSS-signed authorization	P3
OI2-06	PX4 `v1.18.0` not released; P2 is pinned to `v1.18.0-rc1`	P1 DD-06 re-pin
OI2-07	How to recover the DEBIX (boot-mode selection, USB serial download) is not confirmed	Before any fuse is burned
OI2-08	Vendor sources disagree on the DEBIX header pins for ECSPI2; the 3.3 V pin is not confirmed	Schematic or a meter, before the TPM is powered

From P2 `docs/02-cc-boot-chain.md` §2.7, `docs/03-measured-boot-sealing.md` §3.9 and `docs/04-fc-secure-boot.md` §4.6. OI2-03 was closed by DD2-01.

2.7 Rebuilding and rerunning the tests

This section rebuilds P1's and P2's checks and the emulated VE-07 run on a clean Ubuntu 24.04, step by step, with the output to expect. It was followed literally on 2026-10-13, with each command block taken from this document (checked to match its PDF text line for line, so that copying from the PDF gives the same commands) and run in order in one shell, on a WSL 2 distribution freshly imported from the Ubuntu 24.04.5 base tarball; what that run showed is stated with each step. Expected outputs are as of P1 commit `e539aa6` and P2 commit `db31c14`; later commits may print other counts.

What you need. An x86-64 machine with Ubuntu 24.04 (the GitHub runner image is Ubuntu 24.04, and P2's package pins are for it), and network access to `cdimage.ubuntu.com`, the Ubuntu archive and its snapshot server (`archive.ubuntu.com`, `snapshot.ubuntu.com`), Launchpad, `ftp.denx.de` and GitHub. Run as root, or as a user with `sudo`. The PX4 build (TC-11) also needs Docker. The verification run used a WSL 2 distribution with 16 cores and 7.6 GB of memory; steps 1 to 8 (without Docker) used about 3 GB of disk, and steps 2 to 7 took about 9 minutes, 5 of them installing packages.

Step 1: a clean Ubuntu 24.04

On Windows, import the Ubuntu base tarball as a new WSL 2 distribution (PowerShell; paths are examples):

```
# Ubuntu 24.04.5 base image and its checksum list from cdimage.ubuntu.com
$base = "https://cdimage.ubuntu.com/ubuntu-base/releases/24.04/release"
curl.exe -LO "$base/ubuntu-base-24.04.5-base-amd64.tar.gz"
curl.exe -LO "$base/SHA256SUMS"
(Get-FileHash ubuntu-base-24.04.5-base-amd64.tar.gz -Algorithm SHA256).Hash.ToLower()
Select-String -Path SHA256SUMS -Pattern ubuntu-base-24.04.5-base-amd64.tar.gz -SimpleMatch
wsl.exe --import p2-clean C:\wsl\p2-clean ubuntu-base-24.04.5-base-amd64.tar.gz --version 2
wsl.exe -d p2-clean
```

Expected: the hash on the third line equals the one on the `SHA256SUMS` line that `Select-String` prints (`e77b6f10c2590cef872b33ee9f635a0e3fd1f57fb074c0e52b5c7f56147a0c86` for the file used here); the import prints `The operation completed successfully.`, creating the folder if it does not exist; and the last command opens a root shell (`root@<your machine>`). Everything from step 2 on is typed in that shell. On a Linux machine, start from a fresh Ubuntu 24.04 install or container instead.

`SHA256SUMS` only helps if it is genuine. If you have GnuPG, also download `SHA256SUMS.gpg` from the same folder and check it with `gpg --verify SHA256SUMS.gpg SHA256SUMS` against Ubuntu's CD image signing key; in the verification run it reported a good signature from "Ubuntu CD Image Automatic Signing Key (2012)", primary key fingerprint `8439 38DF 228D 22F7 B374 2BC0 D94A A3F0 EFE2 1092`.

Step 2: base tools

The base image has no Python, git, curl, bzip2 or certificates. Install them from the Ubuntu archive (the GitHub runner image has them preinstalled, so P2's package pins do not list them):

```
apt-get update
DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
  ca-certificates git python3 curl bzip2
python3 --version
```

Expected: `Python 3.12.3`. (P1 and P2 need Python 3.11 or later; both use the standard library only.)

Step 3: P1 and its checks

```
cd ~
git clone https://github.com/samyoon727ca/uas-sec-p1-architecture.git p1
cd p1 && git checkout e539aa638d3e44c1982341e927d974b4396e0576
python3 -m unittest discover -s tests
python3 tools/validate_trace.py
```

```
python3 tools/render_views.py --check
python3 tools/gen_sysml.py --check
```

Expected: the unit tests end with `OK (Ran 48 tests` at `e539aa6`); the validator prints `46 elements, 66 threats (3 accepted), 45 requirements, 108 trace links, 17 verification events` and `PASS: 0 error(s), 0 warning(s)`; the view check prints `views up to date`; the SysML check prints `SysML security model up to date; every element has an alias`. Each command exits 0. (The SysML model's own validation, `tools/sysml/validate.sh`, needs Java 21 and is not part of this guide.)

Step 4: P2 and its checks

```
cd ~
git clone https://github.com/samyoon727ca/uas-sec-p2-verified-boot.git p2
cd p2 && git checkout db31c1451c7af859ae08cc2d4b073980cc666a1a
python3 -m unittest discover -s tests
python3 tools/validate_trace.py
python3 tools/render_views.py --check
python3 tools/check_no_secrets.py
python3 tools/check_p1_snapshot.py
```

Expected: the unit tests end with `OK (Ran 121 tests` at `db31c14`; one test also prints `views up to date`); the validator prints `P1 parents SR-005, SR-010, SR-018 (VE-07, VE-08); 5 child requirements, 7 trace links, 14 test cases` and `PASS: 0 error(s)`; `views up to date`; `PASS: 0 file(s) with private key material`; and four `OK` lines followed by `PASS: data/p1 vs samyoon727ca/uas-sec-p1-architecture@e539aa6`. The last check fetches P1's CSVs from GitHub.

Step 5: the pinned host packages

```
cd ~/p2
cc/emu/install-host-packages.sh
```

The script installs QEMU, swtpm, tpm2-tools, the arm64 cross compiler, dtc and the other packages listed in `cc/emu/host-packages.txt` at their exact versions, from the Ubuntu archive snapshot named there, which apt verifies against Ubuntu's signing key (P2 `cc/emu/install-host-packages.sh`). It took about five minutes (288 seconds) in the verification run, most of it downloading from the snapshot server. In a minimal image, with no systemd running, apt also prints warnings from systemd's `tmpfiles` setup, such as `Failed to resolve group 'kvm'`, `Failed to resolve user 'tss'` and `Failed to send reload request`; they did not affect any later step. Expected at the end: one line `OK <package> <version>` per pinned package (20 at `db31c14`), for example `OK qemu-system-arm 1:8.2.2+ds-0ubuntu1.18` and `OK swtpm 0.7.3-0ubuntu5.24.04.1`, and exit status 0. A `DIFF` line means a package is not at its pin; a CI run with a `DIFF` would not count as evidence.

Step 6: build the emulated chain

```
cc/build.sh
```

This downloads the pinned U-Boot `v2026.07` tarball and the arm64 guest packages, refusing any file whose SHA-256 differs from its pin; builds six U-Boot variants (hardened, stock, two control builds, two TC-06 test builds) and checks each `.config` against its fragments; and builds the unseal agent, its control build and two initramfs images (P2 `cc/build.sh`:1–14). Expected: the script ends by printing the SHA-256 of each built file (the six U-Boot binaries and their `.config` files, `Image.gz`, both initramfs images, both unseal agents, `mkimage` and `mkenvimage`), and exits 0. It took 96 seconds in the verification run. The files the PCR prediction is computed from have these digests in the CI evidence:

Built file	SHA-256 in the CI evidence
`u-boot-hardened.bin`	`ef5af6bd69f635b6ded028b649088de240666b859803facb11f4c8bfc11ba1fe`
`Image.gz`	`6001f54164a5201cd5a80e0a09bb05c43fa6e910a0abfe0f5a4e8d7d781cc737`
`initramfs.cpio.gz`	`e690e2cabc09633434334a203cd2ed7263b6ecb37119d2de77974afe1fb0f078`

In the verification run, `Image.gz` (the pinned Ubuntu kernel, extracted unchanged), both initramfs images, both unseal agents, `mkenvimage` and all six `.config` files had exactly the CI digests. The six U-Boot binaries and `mkimage` did not:

- `mkimage` holds the absolute path of the U-Boot source it was built from (a C `__FILE__` string), so its digest changes with the build folder; rebuilding into another folder changed it and nothing else.
- The U-Boot binaries were identical in two fresh distributions and when built in another folder, so their difference from CI's is neither randomness nor the build path. The cause is not known. Every package the local build recorded in `build/toolchain.txt` has the same version in the CI run's list; the CI runner also has 20 other compiler packages installed (such as `gcc-12`, `gcc-14` and `g++`) that a fresh Ubuntu does not have.

That does not change the PCRs: from the U-Boot binary, U-Boot measures only its version string, which is fixed by `source_date_epoch` in `cc/uboot/pin.txt`, and its compiled-in command line, and step 7 shows the predicted PCRs equal to CI's.

Step 7: run the VE-07 cases

```
python3 tests/ve07/boot_cases.py --build build --out /tmp/ve07
python3 tools/check_no_secrets.py --root /tmp/ve07
```

The second command is the check P2's CI runs on its evidence before uploading it (P2 `.github/workflows/ve07-emu.yml`). Expected: one line `PASS <run id> <title> (<seconds>s, <how it ended>)` for each run listed in "The current CI evidence" in §2.5 except TC-04a, the configuration check, which prints no line; then a final line `PASS: 31/31 runs; TC-01 passed, TC-03 passed, TC-04 passed, TC-05 passed, TC-06 passed, TC-07 passed, TC-08 passed, TC-09 passed, TC-10 passed`, and exit status 0; then `PASS: 0 file(s) with private key material`. It took 159 seconds in the verification run. `/tmp/ve07/summary.md` then says "Where: local run (not CI evidence)" and "Host packages match the pins (snapshot 20261002T000000Z): yes".

Compare its TC-10 table with the CI values in "Measured boot: PCRs, extend and the event log" in §2.2. In the verification run all four policy PCRs were equal to CI's, and so was the approved-policy digest for epoch 0 printed under TC-05 (`6a942643065afc6aee87ab94c9b0148a6702f71e89164fbd03e7fdc5a64440d6`, of which `summary.md` shows the first 16 digits), because both depend only on the pinned inputs. The keys' SHA-256 values, the public-key digests and the sealed objects are generated per run and differ from CI's.

Step 8: the FC case (TC-11) needs Docker

TC-11 builds PX4 inside PX4's build container, so it needs Docker (P2 `tests/ve08/README.md`). With Docker installed:

```
cd ~/p2
cc/emu/install-host-packages.sh fc/host-packages.txt
fc/clone-px4.sh /tmp/px4
docker run --rm -v "$PWD:/p2:ro" -v /tmp:/work -e PX4=/work/px4 -e KEYS=/work/keys -e OUT=/work/fc \
  "$(awk '$1 == "container:" {print $2}' fc/px4/pin.txt)" bash /p2/fc/build.sh
python3 tests/ve08/image_cases.py --px4 /tmp/px4 --fc /tmp/fc --out /tmp/ve08
python3 tools/check_no_secrets.py --root /tmp/ve08
```

These are the commands of P2's `ve08-host` workflow, with `/tmp` in place of the runner's temporary folder (P2 `.github/workflows/ve08-host.yml`). Expected: `fc/clone-px4.sh` prints `PX4 v1.18.0-rc1 at fca3df865af36124a28c9d607e850f111dbaaea9`; the container build ends by printing the SHA-256 of `bootloader.bin`, the firmware images and the public key (`fc/build.sh` deletes the per-run private key); the checker prints one `PASS` line per run and ends `PASS: 5/5 runs, 4/4 static checks`, as in the CI evidence ("The current CI evidence" in §2.5); and the last check prints `PASS: 0 file(s) with private key material`. The per-run key makes every image digest differ from CI's.

Installing the FC packages removes the arm64 cross compiler. Ubuntu's `gcc-multilib`, pinned in `fc/host-packages.txt`, conflicts with `gcc-aarch64-linux-gnu`, so the first command of this step uninstalls the compiler step 5 installed (apt prints `Removing gcc-aarch64-linux-gnu`). P2's CI never meets this, because VE-07 and VE-08 run in

separate jobs on fresh runners. To rebuild the CC chain (step 6) after this step, run step 5 again: it reinstalls the cross compiler and removes `gcc-multilib`, and it took 5 seconds with the packages already downloaded.

Not verified locally: the machine used to verify this guide has no Docker. There, the package step printed four `OK` lines and the clone printed the expected line, then `docker: command not found`, and the checker stopped with `FileNotFoundError: … /tmp/fc/k-fc-fw-dev.pub` because nothing was built. The secret check then printed `PASS`, but only because `/tmp/ve08` did not exist: it says nothing in that case. TC-11's only evidence is the CI run.

Step 9: what the rebuild does and does not give you

A clean local run of steps 1 to 7 reproduces the emulated VE-07 results on your machine; it is marked `local`, so it is not CI evidence, and it shows nothing that the CI run does not. Nothing in this guide touches hardware or fuses. The hardware test cases need the DEBIX board, an SPI TPM module, a Pixhawk 6X and an SWD probe; at `db31c14`, P2's tree holds only the emulated and host harnesses (`tests/ve07/`, `tests/ve08/`), no hardware-run scripts.

When you are done, `wsl.exe --unregister p2-clean` (in PowerShell) deletes the distribution and everything in it.

3 Roadmap: P3 to P6

P3 to P6 are planned; none has produced evidence yet. Each section below says what the project is for, lists the P1 requirements and verification events it owns (generated from P1's `data/requirements.csv` and `data/verification.csv`, with parent threats from `data/trace.csv`), and what it inherits from P2, citing the P2 document that hands it over. P1 also names a P7, which fuzzes the CC's command filter (P1 `docs/04-architecture.md` DD-04); it owns no P1 requirement and has no section here.

Each project has one source file, `overview/src/pN.md`. While its front matter says `kind: stub`, the file is a section of this chapter and the tables are added automatically. When the project has its own results, its file becomes `kind: chapter`, is written as a full chapter like P1's and P2's, and leaves this chapter; when no stub is left, the chapter disappears (`overview/README.md`).

3.1 P3: PKI, key management and signed updates; SITL harness

P3 owns the project's keys and signing: the MSS's hardware security module (HSM) and offline backup, MAVLink key provisioning and rotation, signed and non-rollback CC updates, and encryption of imagery and flight logs to MSS-held keys (P1 `docs/07-requirements.md` §7.1; P1 `README.md`). It also hosts the PX4 software-in-the-loop (SITL) harness for VE-01, VE-02 and VE-17, because it owns MAVLink key provisioning (P1 `docs/08-verification-plan.md`). VE-02 closes P1's open item OI-01: until it passes, end-to-end MAVLink signing from GCS to FC is design intent only (P1 `docs/04-architecture.md` §4.5). P3 owns 14 requirements.

Inherited from P2.

- **Custody of P2's keys** (P2 `docs/05-key-management.md` §5.1, §5.3). P3 generates production keys matching P2's key table, in an offline root tier (K-CC-ROOT, K-FC-REC) and an HSM-held intermediate tier (K-CC-OS, K-CC-POL, K-FC-FW), and swaps each file-based signer for an HSM signer: `mkimage -N pkcs11` for FIT configurations and a PKCS#11 ECDSA signer for the approved policies, so that K-CC-POL, the key that decides which CC boot states get the keys, moves into the HSM (P2 `docs/03-measured-boot-sealing.md` §3.3, §3.5).
- **OI2-04:** PX4's `sign_firmware.py` needs the raw Ed25519 private key; SR-034 needs signing inside the HSM, so P3 needs a PKCS#11 Ed25519 signer that writes the same 64-byte signature PX4 appends (P2 `docs/04-fc-secure-boot.md` §4.6).
- **OI2-05:** committing a new anti-rollback epoch uses TPM owner authorization in P2; a fielded CC needs an MSS-signed authorization so that no long-lived secret is stored on the CC (P2 `docs/03-measured-boot-sealing.md` §3.4, §3.9).
- **Update acceptance (SR-007):** each CC release ships its signed approved policy; P3 includes it in the signed update metadata and owns update acceptance, and shares the A/B layout that lets a CC fall back when a new chain cannot unseal with P4 (P2 `docs/03-measured-boot-sealing.md` §3.5; `docs/05-key-management.md` §5.3).
- **FC rollback (THR-066):** P2 found that PX4 Bootloader Secure Boot never checks a version; P1 added THR-066 and SR-045, verified by P3's VE-17, in which the GCS refuses to arm a vehicle whose FC reports older firmware (P2 `docs/01-scope-and-trace.md` §1.4, change 2).

ID	Requirement	On	Method	Verified by	Parent threats
SR-001	The CC shall discard all traffic on its radio interface except WireGuard traffic from the provisioned GCS peer.	CC	Test	VE-01	THR-001, THR-006, THR-010, THR-014, THR-042, THR-043, THR-046, THR-048, THR-064, THR-065
SR-002	The GCS shall discard all traffic on its radio interface except WireGuard traffic from the provisioned CC peer.	GCS	Test	VE-01	THR-015, THR-043, THR-045, THR-046, THR-048

ID	Requirement	On	Method	Verified by	Parent threats
SR-003	The FC shall reject MAVLink messages that are unsigned or not signed with the provisioned key, except HEARTBEAT, RADIO_STATUS, ADSB_VEHICLE and COLLISION.	FC	Demonstration	VE-02	THR-042, THR-049, THR-064
SR-004	The FC shall enter Return mode when no GCS heartbeat has been received for COM_DL_LOSS_T = 10 s (NAV_DLL_ACT = 2).	FC	Test	VE-01	THR-009, THR-044
SR-007	The CC shall install only update images that are signed with the MSS release key and newer than the installed version.	CC	Test	VE-03	THR-008, THR-041, THR-059
SR-015	The CC shall verify the signature of MAVLink frames received from the FC before using them for autonomy decisions.	CC	Test	VE-04	THR-052
SR-017	The CC shall encrypt stored mission imagery to a public key whose private key is held only on the MSS.	CC	Demonstration	VE-05	THR-036
SR-019	The FC shall encrypt flight logs with PX4 log encryption, using a public key whose private key is held only on the MSS.	FC	Demonstration	VE-05	THR-028
SR-025	The GCS shall alert the operator when FC telemetry is unsigned or not signed with the provisioned key.	GCS	Demonstration	VE-02	THR-027, THR-045, THR-057
SR-034	The MSS shall hold software signing private keys in a hardware security module that does not export them.	MSS	Demonstration	VE-06	THR-020, THR-023, THR-039
SR-035	The MSS shall keep an encrypted offline backup of each software signing key.	MSS	Demonstration	VE-06	THR-040
SR-036	The MSS shall provision a new MAVLink signing key to the FC, CC and GCS after any loss of physical custody of a key holder.	MSS	Demonstration	VE-06	THR-007, THR-016, THR-026, THR-034, THR-037
SR-037	The MSS shall provision the first signing key on the FC over USB at first power-up, before any other link is connected.	MSS	Demonstration	VE-06	THR-057
SR-045	The GCS shall alert the operator and refuse to arm the vehicle when the FC reports firmware older than the minimum approved release.	GCS	Test	VE-17	THR-066

The P1 requirements whose evidence repo is P3 (P1 `data/requirements.csv`, `data/trace.csv`).

Event	Title	Method	Verifies	P1 status	Pass criteria
VE-01	Spoofed GCS heartbeats do not suppress the data-link-loss failsafe	Test	SR-001, SR-002, SR-004	planned	FC enters Return within COM_DL_LOSS_T plus a stated tolerance; CC and GCS counters show injected packets dropped; control run shows the failsafe suppressed.
VE-02	Signed C2 works with the pinned versions	Demonstration	SR-003, SR-025	planned	Signed commands execute; unsigned and wrongly signed commands have no effect; the GCS alerts on unsigned telemetry. Closes OI-01.
VE-03	CC accepts only signed, newer update images	Test	SR-007	planned	Only the valid newer image installs; each rejection is logged.
VE-04	CC autonomy ignores unsigned or altered FC telemetry	Test	SR-015	planned	Autonomy decisions use only frames with valid signatures; injected frames are logged and discarded.
VE-05	Imagery and flight logs are unreadable without the MSS key	Demonstration	SR-017, SR-019	planned	Copies from the vehicle are unreadable; the MSS decrypts both.
VE-06	Key management on the MSS	Demonstration	SR-034, SR-035, SR-036, SR-037	planned	Export fails; the restored key signs an artifact that verifies; after rotation the old key is rejected by FC, CC and GCS; first-key provisioning follows the defined order.
VE-17	GCS refuses to arm a vehicle with older FC firmware	Test	SR-045	planned	With the older version the GCS alerts the operator and the vehicle stays disarmed; with the approved version it arms.

The P1 verification events P3 runs (P1 `data/verification.csv`).

3.2 P4: Hardened CC image, MSS pipeline and supply chain

P4 builds what runs above the boot chain and the pipeline that builds it: the CC's operating-system image (Yocto or Buildroot, P1 `docs/01-system-description.md` §1.4) with a read-only, integrity-checked root filesystem, a single service holding the signing key, confined parsers, a USB allowlist, a hardware watchdog and authenticated maintenance logins; the CC's MAVLink command filter and its logging (DD-04); and the MSS build pipeline with hash-pinned dependencies, SBOMs, vulnerability scans and build provenance (P1 `README.md`; `docs/07-requirements.md` §7.2). P4 owns 14 requirements.

Inherited from P2.

- **The root filesystem (SR-006).** P2's chain ends at the kernel and initramfs; the root filesystem is verified by the kernel with dm-verity, which is P4's (P2 `docs/02-cc-boot-chain.md` §2.2; `docs/01-scope-and-trace.md` §1.5).
- **Runtime integrity.** P2 seals the keys to PCRs 0, 1, 8 and 9 only; PCRs from 10 up change at runtime (IMA, for example) and runtime integrity is left to P4 (P2 `docs/03-measured-boot-sealing.md` §3.1).
- **The A/B layout,** shared with P3, that lets a CC boot its previous chain when a new one cannot unseal (P2 `docs/03-measured-boot-sealing.md` §3.5).
- **The DEBIX image.** P2 boots a busybox initramfs; the hardened image that P4 builds is what the unsealed keys are handed to. P2's hardened U-Boot configuration and its DEBIX fragment (P2 `docs/02-cc-boot-chain.md` §2.4) are inputs to that image's boot loader. *Assumption: P4 keeps U-Boot `v2026.07`, so that P2's PCR prediction carries over; P2's documents do not state this.*

ID	Requirement	On	Method	Verified by	Parent threats
SR-006	The CC shall mount its root filesystem read-only with block-level integrity verification.	CC	Test	VE-09	THR-008, THR-032
SR-008	The CC shall restrict read access to the MAVLink signing key to a single signing service.	CC	Test	VE-09	THR-010
SR-009	The CC shall run the MAVLink router, the command filter and the video pipeline as unprivileged processes confined by a mandatory access control policy.	CC	Test	VE-09	THR-010, THR-014, THR-053
SR-011	The CC shall forward from the GCS tunnel to the FC only messages and commands on the approved allowlist, which excludes SERIAL_CONTROL, FILE_TRANSFER_PROTOCOL, SETUP_SIGNING, RADIO_STATUS, ADSB_VEHICLE and COLLISION.	CC	Test	VE-10	THR-003, THR-029, THR-050, THR-051, THR-057
SR-012	The CC shall rate-limit MAVLink traffic forwarded to the FC to the approved per-message rates.	CC	Test	VE-10	THR-005
SR-013	The CC shall log each command it forwards to the FC with a timestamp and either the sending WireGuard peer or a CC-originated marker.	CC	Test	VE-10	THR-004
SR-014	The CC shall accept only the USB device classes and identifiers on the approved payload list.	CC	Test	VE-09	THR-011, THR-053
SR-016	The CC shall restore C2 forwarding through a hardware watchdog when its C2 services stop responding.	CC	Test	VE-09	THR-009
SR-028	The GCS shall install only software packages whose signatures verify against the MSS release key.	GCS	Test	VE-11	THR-017, THR-019
SR-031	The MSS shall build release artifacts only from dependencies pinned by cryptographic hash to approved sources.	MSS	Test	VE-11	THR-021, THR-025, THR-063
SR-032	The MSS shall produce an SBOM and a vulnerability scan for every release artifact, including PX4 firmware.	MSS	Test	VE-11	THR-006, THR-063
SR-033	The MSS shall record build provenance for every release artifact and verify it before loading.	MSS	Test	VE-11	THR-021, THR-041, THR-063
SR-038	The MSS shall process offloaded vehicle data in an account that has no access to key material.	MSS	Test	VE-11	THR-061, THR-062
SR-044	The CC shall accept maintenance connections only when authenticated with the MSS maintenance key.	CC	Test	VE-09	THR-059

The P1 requirements whose evidence repo is P4 (P1 `data/requirements.csv`, `data/trace.csv`).

Event	Title	Method	Verifies	P1 status	Pass criteria
VE-09	CC image hardening	Test	SR-006, SR-008, SR-009, SR-014, SR-016, SR-044	planned	Modified blocks fail to read; only the signing service reads the key; confined services cannot reach credentials; the device is refused; the watchdog restores C2 forwarding; the login is refused.
VE-10	CC command filter and logging	Test	SR-011, SR-012, SR-013	planned	Excluded messages never reach the FC; the approved command does; the flood is limited to the approved rate; each forwarded command is logged with time and WireGuard peer.
VE-11	Supply-chain pipeline and package integrity	Test	SR-028, SR-031, SR-032, SR-033, SR-038	planned	Both bad builds fail; the release has an SBOM, a vulnerability scan and provenance; the tampered artifact and unsigned package are rejected; the offload account cannot read key material.

The P1 verification events P4 runs (P1 `data/verification.csv`).

3.3 P5: Hardware and firmware security assessment

P5 tests owned hardware against the physical adversary of P1's tier 2 (assumption A-08): tamper-evident seals over the FC's SD card, debug connectors and wiring, and an FC debug port that cannot be used in the flight configuration (P1 `README.md`; `docs/08-verification-plan.md`). P5 does not need SITL. It owns 2 requirements.

Inherited from P2.

- **Physical attacks on the boot chain and the TPM.** P2 places debug ports, bus interposers and glitching out of its scope and in P5's (P2 `docs/01-scope-and-trace.md` §1.5). Neither CI nor P2's hardware runs can prove resistance to TPM bus interposers, a TPM reset, debug-port access or fault injection (P2 `docs/06-verification.md` §6.2).
- **TPM reset and replay (A2-02).** SR-010.4's encrypted session stops passive reads of the unsealed keys, not an attacker who can reset the TPM on its own and replay approved measurements. P2 assumes the TPM resets only at power-on, with its reset pin on the module's pull-up (DD2-17), to be checked on the module bought; P1 records the remaining risk in THR-034, and it is P5's scope (P2 `docs/03-measured-boot-sealing.md` §3.6; P1 `docs/05-threat-model.md` §5.5).
- **The FC boot loader's root of trust.** The PX4 boot loader sets no flash write-protection, so SR-018 holds only while the debug port is disabled or sealed, which is P5's SR-024 (P2 `docs/04-fc-secure-boot.md` §4.1; SR-018's rationale in P1 `data/requirements.csv`).

ID	Requirement	On	Method	Verified by	Parent threats
SR-023	The FC shall carry tamper-evident seals over its SD-card slot, debug connectors and sensor and actuator wiring that are inspected before each flight.	FC	Inspection	VE-12	THR-013, THR-026, THR-027, THR-049, THR-055, THR-056, THR-065
SR-024	The FC shall have its SWD/JTAG debug port disabled or physically sealed in the flight configuration.	FC	Test	VE-13	THR-030

The P1 requirements whose evidence repo is P5 (P1 `data/requirements.csv`, `data/trace.csv`).

Event	Title	Method	Verifies	P1 status	Pass criteria
VE-12	Tamper-evident seal inspection	Inspection	SR-023	planned	Seals cover every listed access point, and the SD-card access is visibly evident.
VE-13	FC debug port cannot be used	Test	SR-024	planned	No debug session can be established, or the port is sealed so that any attempt is evident.

The P1 verification events P5 runs (P1 `data/verification.csv`).

3.4 P6: RMF-as-code (OSCAL): configuration and procedure evidence

P6 produces configuration and procedure evidence for a Risk Management Framework package, written as code in OSCAL: inspections of the FC's parameter baseline (GNSS spoofing and jamming checks, ADS-B response, disabled uXRCE-DDS and spare MAVLink instances), of the GCS (individual logins, disk encryption, a single enabled network interface, sortie records) and of the MSS (logins, maintenance and provisioning logs, peripheral firmware versions, wired and offline key handling) (P1 `README.md`; `docs/07-requirements.md` §7.1). P6 owns 12 requirements.

Inherited from P2. P2's documents at `db31c14` hand nothing to P6 by name. *Assumption, not stated by P2: an RMF package would cite P2's evidence for SR-005, SR-010 and SR-018 once P1 records VE-07 and VE-08 as passed.*

ID	Requirement	On	Method	Verified by	Parent threats
SR-020	The FC shall run with the GNSS spoofing and jamming checks enabled (EKF2_GPS_CHECK bits 9 and 11 set).	FC	Inspection	VE-14	THR-012, THR-024, THR-054
SR-021	The FC shall take no automatic flight action on ADS-B traffic reports (NAV_TRAFF_AVOID = 0 or 1).	FC	Inspection	VE-14	THR-050
SR-022	The FC shall have only the MAVLink instance for the CC link enabled besides USB, with the uXRCE-DDS client disabled (UXRCE_DDS_CFG = 0).	FC	Inspection	VE-14	THR-002
SR-026	The GCS shall require individual operator authentication to unlock a session.	GCS	Inspection	VE-15	THR-016, THR-018
SR-027	The GCS shall use full-disk encryption.	GCS	Inspection	VE-15	THR-016, THR-037, THR-038
SR-029	The GCS shall have no network interface enabled other than the ground-radio link during operations.	GCS	Inspection	VE-15	THR-019
SR-030	The GCS shall record each sortie's sent and received MAVLink traffic with the operator account that ran it.	GCS	Inspection	VE-15	THR-004, THR-018
SR-039	The MSS shall log each software load and key-provisioning action with the maintainer's identity.	MSS	Inspection	VE-16	THR-022
SR-040	The MSS shall require individual maintainer authentication.	MSS	Inspection	VE-16	THR-020, THR-022
SR-041	The MSS shall record the firmware versions of the GNSS module, ESCs, radios and payload at each maintenance action and compare them with the approved baseline.	MSS	Inspection	VE-16	THR-011, THR-012, THR-013
SR-042	The MSS shall provision GCS keys over a direct wired connection and never through removable media.	MSS	Inspection	VE-16	THR-060
SR-043	The MSS shall generate and provision keys only while disconnected from external networks.	MSS	Inspection	VE-16	THR-023, THR-039

The P1 requirements whose evidence repo is P6 (P1 `data/requirements.csv`, `data/trace.csv`).

Event	Title	Method	Verifies	P1 status	Pass criteria
VE-14	FC parameter baseline	Inspection	SR-020, SR-021, SR-022	planned	Bits 9 and 11 are set; NAV_TRAFF_AVOID is 0 or 1; uXRCE-DDS is disabled; only the CC-link MAVLink instance is enabled besides USB.
VE-15	GCS configuration	Inspection	SR-026, SR-027, SR-029, SR-030	planned	Individual logins and screen lock are enforced; the disk is encrypted; only the ground-radio interface is enabled; each sortie record names its operator.
VE-16	MSS configuration and records	Inspection	SR-039, SR-040, SR-041, SR-042, SR-043	planned	Individual logins; every load and provisioning action is logged with a maintainer; peripheral versions are recorded against the baseline; GCS keys are provisioned wired; key operations happen offline.

The P1 verification events P6 runs (P1 `data/verification.csv`).

4 Limitations and open findings

This chapter reports an independent, adversarial audit of P1 and P2, made on 2026-10-13 at the commits below. It was made by a workstream separate from the one that wrote the rest of this document, and the two did not share drafts until this chapter was written. The findings are reproduced exactly as the audit wrote them. Every table here is generated from a byte-for-byte copy of the audit's `findings.json` (`overview/audit/`), so nothing in this chapter is reworded, shortened or summarized. None of the findings has been triaged or fixed: each stays open until the owner closes it in P1 or P2.

Item	Value
Audit	P1 and P2, adversarial, read-only, 2026-10-13
P1 audited at	`e539aa638d3e44c1982341e927d974b4396e0576`
P2 audited at	`db31c14 (origin/main), P1 snapshot pinned to e539aa6`
High (2)	A-01, A-02
Medium (14)	A-03, A-04, A-05, A-06, A-07, A-08, A-09, A-10, A-11, A-12, A-13, A-14, A-18, A-24
Low (18)	A-15, A-16, A-17, A-19, A-20, A-21, A-22, A-23, A-25, A-26, A-27, A-28, A-29, A-30, A-31, A-32, A-33, A-34
Info (1)	A-35

The audit's baselines and its findings by severity (`overview/audit/findings.json`).

4.1 What the audit did

It read both repositories at the commits above. It went through P2 in depth, P1 at the level of scope, purpose and plan quality, and the seam between them. It then tested P2's test cases by mutation: in a WSL Ubuntu 24.04 distro with P2's pinned host packages, it broke one thing at a time and re-ran the harness. Some changes were made consistently in two places, where the question was whether shared constants hide a fault. The things broken were the U-Boot configuration or code, the unseal agent, the MSS policy tool, the provisioning and the harness. A test case that stays green when the property it claims is broken has a gap.

- **VE-07.** 18 experiments, plus a full baseline run of all 31 runs, which passed.
- **TC-11.** 5 experiments on the host checker, with synthetic images, because the PX4 build needs Docker.
- **Validator and evidence.** 4 experiments on P2's validator and secret scan.

The audit also checked the evidence against GitHub: it downloaded each run's artifact and compared it byte for byte with the committed folder, and it compared a local rebuild with CI. And it re-checked the findings of the earlier audit of 2026-10-09.

The audit's full report and its experiment scripts and results are held with the owner for triage, outside this repository. "Evidence" cells that name `experiments/...` refer to that folder. In those cells, **[verified]** means checked by reading code or data or by an experiment, and **[analysis]** means reasoned from a specification or source without a demonstration.

Severity	Meaning
High	a security property or recorded pass does not hold as written, or holds only under a condition nobody owns
Medium	a test cannot detect a failure it is meant to detect, a seam leaves an obligation unowned, or evidence integrity depends on something unchecked
Low	stale, inconsistent or unclear text; reproducibility friction; hygiene
Info	checked and sound

Severity scale (`overview/audit/findings.json`).

What this changes in the rest of this document. Where a finding narrows what a test case's pass shows, the test case's section in chapter 2 now links to it:

- TC-04: A-06;
- TC-05 and TC-09: A-04, A-10;
- TC-06 and TC-10: A-03;
- TC-07: A-25;
- TC-08: A-01, A-05;
- TC-11: A-11;
- TC-14: A-24.

The concept sections on release policies, the epoch and key release link to A-01, A-02 and A-05. Statements that P2's documents make, and that this document quotes, are left as P2 states them; the findings say where they overstate.

4.2 Index of findings

ID	Severity	Repo	Finding
A-01	high	P2	Anti-rollback epoch can be reset by anyone holding the (empty) TPM owner authorization
A-02	high	P2, P1 (seam with P4)	PCRs are never capped after unseal; keys stay releasable for the rest of an approved boot
A-03	medium	P2	TC-06's kernel variant changes only the gzip header; a shared measurement fault passes every case
A-04	medium	P2	TC-09 cannot detect an unsalted unseal session, which SR-010.4 forbids
A-05	medium	P2	The policyRef binding is untested; emptying it leaves every case green
A-06	medium	P2	TC-04a's .config check is circular; the DEBIX environment case targets the wrong location
A-07	medium	P2	The validator accepts any existing file as evidence for a pass
A-08	medium	P2	Evidence provenance expires with the GitHub artifacts (2027-01-09 to 01-11); no digests recorded
A-09	medium	P1 (seam with P2 and P5)	P1 VE-13 passes with a working but sealed FC debug port, which SR-018 depends on
A-10	medium	P2	SR-010.4 is verified only on swtpm's log, with no hardware run, contrary to A2-04
A-11	medium	P2	TC-11 re-implements PX4's boot decision and fixes flash at 2 MiB; the copies already diverge
A-12	medium	P2 (seam with P3, P4)	A2-01 (every signed U-Boot measures) has no owner and no control
A-13	medium	P1 (seam with P2 and P3)	P2's hand-offs to P3 (OI2-04, OI2-05, K-CC-POL) are not recorded in P1
A-14	medium	P2, P1 (seam with P3)	No path to put a new or MSS-generated key into a fielded CC's TPM (SR-036)
A-18	medium	P2 (seam with P3)	TPM reset only at power-on conflicts with the update flow's 'Reboot'; emulation hides it
A-24	medium	P2	DD2-14's recovery key is neither built nor tested; TC-14 cannot fail as written
A-15	low	P2	docs/06 says every CI item is 'proved', beyond what the tests show
A-16	low	P2	U-Boot binaries are not bit-reproducible between CI and a pinned local host
A-17	low	P2	The VE-07 and VE-08 host package lists cannot coexist on one host
A-19	low	P2	docs/01 names P1 commit 83c6d2d; the snapshot is at e539aa6
A-20	low	P2	Evidence column: folder in docs/06, file in the CSV
A-21	low	P2	tests/ve07/README: stale variant count; local runs written into evidence/ve07-ci/
A-22	low	P2	docs/05 says PX4's test key is used only in TC-13; TC-11 uses it too
A-23	low	P2	docs/03 says TC-05 to TC-10 reproduced all prototype findings; two are not
A-25	low	P2	The parent-name check trusts a file on unprotected storage; its purpose is untested
A-26	low	P2	CI path filters miss files the harnesses import or run
A-27	low	P2	The secret scanner's fallback flags its own compiled test (2026-10-09 P2-13)
A-28	low	P2	Stale reference link, misdescribed snapshot mechanism, unexplained U-Boot log lines
A-29	low	P1, P2	Assumptions in P1 and P2 have no owner or retiring test
A-30	low	P1	P7 is assigned work in P1 but not defined or allocated

ID	Severity	Repo	Finding
A-31	low	P1	P1 findings of 2026-10-09 still open (P1-01, -02, -06 to -15)
A-32	low	P1	Several P1 verification events have criteria that cannot fail
A-33	low	P1	P1 has no .gitattributes; Windows checkouts get CRLF
A-34	low	P2	The harness's 'modified' flag ignores tools/, which it imports
A-35	info	P2	Checked and sound: evidence, snapshot, baseline, most mutations

4.3 Findings

A-01: Anti-rollback epoch can be reset by anyone holding the (empty) TPM owner authorization

A-01	High
Repo	P2
Where	docs/03-measured-boot-sealing.md:63 'In P2 the epoch bit is set with TPM owner authorization. A fielded CC needs an MSS-signed authorization instead (OI2-05, P3)'; docs/02-cc-boot-chain.md:83 'So a rolled-back CC cannot join C2'; tests/ve07/provision.py:97 'owner authorization available (empty in P2)'; tests/ve07/provision.py:47 EPOCH_NV_ATTRIBUTES = 'nt=bits|ownerwrite|authread|no_da'
Finding	SR-010.3 is P2's only anti-rollback (DD2-07). It holds only while the epoch index keeps its bits. The index is defined in the owner hierarchy without TPMA_NV_POLICY_DELETE, so any caller with owner authorization can remove it, and P2 leaves owner authorization empty; nothing in P2 sets lockout or platform authorization either. The signed policies identify the index by its name, which tools/approve_policy.py:75-80 computes from handle, attributes, size and authPolicy only, so an index defined again with the same public area has the same name, starts with no bits set, and satisfies the policy of every epoch. In P2 that caller can be any code with TPM access on the CC, including a rolled-back chain that the epoch exists to lock out. TC-08 rolls back only an honest older chain that does not touch the TPM, and no case checks that the index cannot be removed. OI2-05 covers advancing the epoch, not removing it.
Evidence	[analysis] TPM 2.0 Library Part 3, TPM2_NV_UndefineSpace authorization (owner or platform; policy-delete indices excluded); P2 code as cited. Not demonstrated in this audit.
Proposed fix	Make removal of the epoch index need an authorization the CC never holds: for example TPMA_NV_POLICY_DELETE with an MSS-authorized policy, or owner, lockout and platform authorization set to values held only by the MSS, with the platform hierarchy disabled or re-randomized by U-Boot at every boot. Extend OI2-05 to cover removal. Add a TC-08 case in which an owner-authorized caller removes and re-creates the index and the older chain must still be refused. Until then reword docs/02:83 and docs/06:24 to state the condition.

A-02: PCRs are never capped after unseal; keys stay releasable for the rest of an approved boot

A-02	High
Repo	P2, P1 (seam with P4)
Where	P1 data/requirements.csv:11 SR-010 rationale 'a modified boot chain cannot unseal them'; P2 docs/03-measured-boot-sealing.md:3 'released ... only to the measured boot state'; cc/initramfs/init:20-36 (no PCR extend after p2-unseal)
Finding	After p2-unseal releases the keys, nothing extends a policy PCR, so the PCRs keep their approved values for the rest of the boot and the same signed policy releases the keys again to any later code. In P2 the only verified and measured code is U-Boot, the kernel, the devicetree and the initramfs. The root filesystem that follows is P4's (SR-006, not built), and the design does not say that SR-006's dm-verity root hash must come from the measured or signed boot. Until both hold, an offline change to the root filesystem (THR-032, which P2 traces to SR-005.1) runs in an approved PCR state with TPM access. Neither P1 nor P2 records SR-010's dependency on SR-006 the way P1 recorded SR-018's on SR-024 (P2 change 3).
Evidence	X01 (experiments/results/X01-second-unseal.txt; console /root/audit/ev/X01-second-unseal/TC-05.log in p2-noble): after the first unseal, a second p2-unseal run in the same boot printed 'wireguard ok' and 'mavlink ok', and the SHA-256 of each key it released equals the sealed key's.
Proposed fix	After unsealing, extend a policy PCR (for example PCR 9) with a fixed separator, so a second unseal in the same boot fails, and add a test case that checks it. Record in P1 (SR-010 or SR-006 rationale) and in P2 docs/01 §1.5 that SR-010's binding extends past the initramfs only once SR-006 is in place with its root hash carried in the signed FIT or the measured command line.

A-03: TC-06's kernel variant changes only the gzip header; a shared measurement fault passes every case

A-03	Medium
Repo	P2
Where	tests/ve07/boot_cases.py:630 'kernel[4:8] = struct.pack("<I", 1) # gzip MTIME: the same kernel, other bytes'; data/testcases.csv:7 TC-06 'Boot chains that each differ in one measured item (... kernel ...)'
Finding	TC-06d changes only bytes 4 to 7 of the stored kernel (the gzip MTIME). A measurement that covers only the start of the kernel image still moves PCR 8 for that change, so TC-06d cannot tell whole-image measurement from partial measurement. TC-10 catches a U-Boot-only fault, because the predictor hashes the whole image (M04a red), but if the predictor shares the fault, every case passes (M04b green). The kernel body is then unmeasured and nothing in VE-07 notices.
Evidence	M04a: U-Boot measures the first 4 KiB of the kernel; TC-05 and TC-10 red. M04b: same, and predict_pcrs.py hashes the same 4 KiB; TC-05, TC-06d and TC-10 all green (experiments/results/M04a-kernel4k-uboot.txt, M04b-kernel4k-both.txt). M15 tests a tail change instead (experiments/results/M15-tc06-tail.txt).
Proposed fix	Make TC-06d change the kernel image away from its header, for example at the tail of the stored image (see M15) or by recompressing at another gzip level so every block changes, and keep the 'only PCR 8 moved' check. Add to TC-10 an independent check that the logged PCR 8 digest equals SHA-256 of the whole kernel data node, computed without predict_pcrs.py.

A-04: TC-09 cannot detect an unsalted unseal session, which SR-010.4 forbids

A-04	Medium
Repo	P2
Where	data/requirements.csv:6 SR-010.4 'The CC shall unseal its keys only over a salted TPM session with response encryption'; data/testcases.csv:10 TC-09 pass criteria 'The unsealed key bytes never appear in the recorded traffic; in the control run they do'; cc/initramfs/p2-unseal.c:126-128
Finding	With the agent's session started unsalted and unbound, and response encryption still on, TC-05 and TC-09 pass. TC-09 is the only test case for SR-010.4, and its criterion checks only that the key bytes do not appear in clear. In TPM 2.0, a session with neither salt nor bind derives its session key only from values that cross the interface, so the salt is what makes the encryption meaningful against a bus probe. The test cannot detect that the salt is gone.
Evidence	M07 (experiments/results/M07-unsalted.txt): Esys_StartAuthSession with tpmKey ESYS_TR_NONE; TC-05 pass, TC-09 pass. [analysis] TPM 2.0 Library Part 1, session key computation for unsalted, unbound sessions.
Proposed fix	Add a check to TC-09 that each policy session used for TPM2_Unseal was started by a TPM2_StartAuthSession naming the persistent parent as tpmKey with a non-empty encryptedSalt, parsed from the recorded commands. Keep an unsalted agent build as a second control that must fail that check.

A-05: The policyRef binding is untested; emptying it leaves every case green

A-05	Medium
Repo	P2
Where	docs/03-measured-boot-sealing.md:33 'policyRef = p2/cc-keys. policyRef names this use, so a policy the MSS signs for another purpose cannot be reused here'; tools/approve_policy.py:44; cc/initramfs/p2-unseal.c:44
Finding	With policyRef emptied in both the MSS tool and the agent, TC-05 and TC-08a to TC-08d all pass. No test case presents a policy that K-CC-POL validly signed under another policyRef, so the binding docs/03 relies on can disappear unnoticed. The unit test keeps the two constants equal (tests/test_approve_policy.py:136), which is consistency, not the property.
Evidence	M11 (experiments/results/M11-policyref.txt): harness exit 0.
Proposed fix	Add a TC-08 case: an approved policy for the current release signed with the real K-CC-POL over a different policyRef must be refused, at PolicyAuthorize.

A-06: TC-04a's .config check is circular; the DEBIX environment case targets the wrong location

A-06	Medium
Repo	P2
Where	docs/02-cc-boot-chain.md:62 '[cc/build.sh] fails if any line is missing from the built .config, and TC-04 checks it again'; docs/02-cc-boot-chain.md:72 'the DEBIX fragment must turn it off, and the .config check must cover it'; data/testcases.csv:5 TC-04 'inject a boot-loader environment with another bootcmd into flash'; cc/uboot/check_config.py:38-40

A-06	Medium
Finding	check_config.py, and TC-04a which reuses it, confirm that the built .config holds the lines of the same fragments it was built from. A hardening line deleted from a fragment is therefore never noticed: with legacy images re-enabled (M01), the flash environment re-enabled (M02) or measured boot off (M03), TC-04a passes and only the behavioral cases fail. In emulation those cases exist. On the DEBIX the defconfig sets ENV_IS_IN_MMC, and TC-04's hardware procedure injects the environment 'into flash', so a DEBIX fragment that forgot to unset ENV_IS_IN_MMC would pass both the .config check and the procedure as written. M02 shows that ENV_IS_NOWHERE does not keep another enabled location from being read.
Evidence	M01, M02, M03 (experiments/results/): TC-04a pass in each; TC-04e, TC-04f, TC-05/TC-10 red respectively. M02 log: 'Loading Environment from Flash... OK', autoboot countdown, injected bootcmd ran.
Proposed fix	Add a fixed policy check, independent of the fragments: no CONFIG_ENV_IS_IN_* other than NOWHERE; LEGACY_IMAGE_FORMAT, BOOTSTD_BOOTCOMMAND and USE_PREBOOT off; FIT_SIGNATURE, MEASURED_BOOT and MEASURE_DEVICETREE on; BOOTDELAY -2. Run it in build.sh and TC-04a. Change TC-04's hardware procedure to inject into the board's own environment location (MMC on the DEBIX).

A-07: The validator accepts any existing file as evidence for a pass

A-07	Medium
Repo	P2
Where	README.md:74 'a passed or failed result has no evidence' (validator rule); tools/validate_trace.py:212-221; docs/06-verification.md:61 'A test case's evidence column points at that folder'
Finding	The validator accepts a passed status if the evidence path exists under evidence/. It does not check that the path is a run folder, that its summary.json records that test case as passed, that its environment is ci for a CI status or hw for a hardware status, or that it belongs to the right event. render_views.py then counts the pass. Prior finding P2-10 (2026-10-09) is still open.
Evidence	D1: TC-12 hw_status=passed with evidence/ve07-ci/20261011-7bc523b/summary.md: validator PASS, generated view 'VE-08 ... 1 of 3' hardware runs passed. D2: TC-05 pointed at a run without TC-05: PASS. D3: evidence/README.md accepted as TC-10 evidence: PASS (experiments/data_mut.py).
Proposed fix	Require the evidence to be a run folder (evidence/<event>-<ci|hw>/<date>-<commit>/) whose summary.json names the same event, has environment.kind matching the column (ci or hw), and records test_cases[tc_id] equal to the CSV status. Add unit tests for D1 to D3. Make docs/06:61 and the CSV agree on folder versus summary.md.

A-08: Evidence provenance expires with the GitHub artifacts (2027-01-09 to 01-11); no digests recorded

A-08	Medium
Repo	P2
Where	evidence/README.md:16 'A ci folder is that artifact, downloaded unchanged'; docs/06-verification.md:66 'A ci folder is the unchanged artifact of a GitHub Actions run'
Finding	True today: each of the six committed folders is byte-identical to its artifact. The artifacts expire between 2027-01-09T07:05Z and 2027-01-11T05:28Z, and GitHub run logs expire with them. The repo records run IDs but not artifact IDs or digests, so after expiry nobody can check the claim, and a folder edited after download could not be told from the original.
Evidence	[verified] gh api .../actions/runs/<id>/artifacts on 2026-10-13: artifact IDs, sha256 digests and expiry for runs 38351585198, 38349138261, 38347668352, 38345035945, 38340963718, 38120187352; gh run download of each, diff -r against the committed folder: identical.
Proposed fix	Record in evidence/README.md, per folder, the artifact ID and the digest GitHub reports, plus a SHA-256 manifest of the folder. For lasting provenance, attest the artifact (actions/attest-build-provenance) or attach it to a release, which does not expire.

A-09: P1 VE-13 passes with a working but sealed FC debug port, which SR-018 depends on

A-09	Medium
Repo	P1 (seam with P2 and P5)
Where	P1 data/verification.csv:14 VE-13 pass criteria 'No debug session can be established, or the port is sealed so that any attempt is evident'; P1 data/requirements.csv:19 SR-018 rationale 'SR-018 holds only while SR-024 keeps the debug port closed'

A-09	Medium
Finding	VE-13 passes if the debug port works but is sealed. SR-018's root of trust is the FC boot loader, which sets no flash write protection, so SR-018 depends on the port being closed, not on tampering being evident. Against a captured vehicle (A-08 tier 2), a seal records the attack but does not stop it, so a VE-13 pass would not support SR-018 as P1 and P2 (docs/04:22) state it.
Evidence	[verified] P1 data as cited; PX4 boot loader sets no write protection (P2 docs/04-fc-secure-boot.md:22).
Proposed fix	Split VE-13's criteria: the pass requires that no debug session can be established in the flight configuration (for example, STM32H7 readout protection at the level that disables debug), and 'sealed' is recorded as a residual risk against SR-018 rather than as a pass.

A-10: SR-010.4 is verified only on swtpm's log, with no hardware run, contrary to A2-04

A-10	Medium
Repo	P2
Where	docs/01-scope-and-trace.md:50 A2-04 'Hardware runs repeat the sealing cases on a real TPM'; tests/ve07/README.md:40 'TC-09 shows that the unsealed keys cross the TPM interface only encrypted; on hardware the same bytes cross SPI'; data/testcases.csv:10 TC-09 hw_status n/a
Finding	TC-09 is the only test case for SR-010.4, the requirement that exists because of a bus probe on a captured vehicle, and it has no hardware run. Its evidence is swtpm's own log of command and response buffers, not a capture of the SPI bus. That the same bytes cross SPI is stated as fact, but it is an assumption. A2-04 says the sealing cases repeat on hardware, but TC-09 does not. Prior finding P2-11 is still open.
Evidence	[verified] data/testcases.csv:10; tests/ve07/boot_cases.py:207-218 parses swtpm log level 20.
Proposed fix	Either add a hardware TC-09 that captures the TPM's SPI lines with a logic analyzer during unseal and searches the capture, or record SR-010.4 as emulation-only with a stated rationale and reword A2-04 and tests/ve07/README.md:40.

A-11: TC-11 re-implements PX4's boot decision and fixes flash at 2 MiB; the copies already diverge

A-11	Medium
Repo	P2
Where	docs/04-fc-secure-boot.md:50 'PX4's own find_toc(), verify_app(), stub keystore and monocypher'; tests/ve08/host/check_image.c:84-110; tests/ve08/host/sw_crypto_ed25519.c:45-62; tests/ve08/host/hw_config.h:14-15
Finding	TC-11 runs PX4's find_toc() and verify_app(), but re-implements jump_to_app()'s loop (PX4 bl.c:308-389) and the Ed25519 path of sw_crypto. The re-implementations differ from PX4. check_image.c omits the first check in bl.c:317, which stays in the boot loader when the app's first word is 0xFFFFFFFF. The crypto stand-in adds a keylen == 32 test that PX4's crypto_signature_check lacks, and reads keys through keystore_get_key rather than PX4's key cache. Nothing checks that these stand-ins still match PX4 at the next pin (A2-05 is about behavior only). The flash size is fixed at 2 MiB here but read from the chip at run time on the board, and it is not labeled as an assumption. The CI image is 1,941,520 bytes, 24,560 bytes short of the 1,966,080 bytes of app flash.
Evidence	[verified] Comparison with PX4 fca3df8 platforms/nuttx/src/bootloader/common/bl.c and src/drivers/sw_crypto/crypto.c; evidence/ve08-ci/20261013-aac4299/TC-11a.log line 1. T series with synthetic images (experiments/results/T-series.txt). T1 and T2: mutated stand-ins turn every negative case red. T3: a validly signed image whose first word is 0xFFFFFFFF 'starts' in the checker, while PX4's boot loader would stay. T4: with BOARD_FLASH_SIZE at 1 MiB, a 1,941,520-byte image cannot be placed (exit 2).
Proposed fix	Compile bl.c's jump_to_app() with stubs, or add a check that the copied logic still matches PX4 at the pin. Label BOARD_FLASH_SIZE as an assumption retired by TC-12, and add a headroom check on image size.

A-12: A2-01 (every signed U-Boot measures) has no owner and no control

A-12	Medium
Repo	P2 (seam with P3, P4)
Where	docs/01-scope-and-trace.md:47 A2-01 'Every u-boot.itb ever signed with K-CC-ROOT contains a U-Boot with measured boot enabled'
Finding	A2-01's own rationale says that if it fails, a signed U-Boot that does not measure lets the kernel extend approved values itself and unseal the keys. No requirement, test case, open item or owner retires or enforces it. cc/build.sh checks the emulated build's .config, not what K-CC-ROOT signs, and HAB signing is an MSS step in P3 or P4.

A-12	Medium
Evidence	[verified] P2 docs/01:43-52 (no retiring TC or OI for A2-01); P2 data/testcases.csv; P1 data/requirements.csv has no MSS requirement on what the boot key may sign.
Proposed fix	Give A2-01 an owner (P3 or P4) and a control: the MSS step that HAB-signs u-boot.itb refuses a build whose .config lacks MEASURED_BOOT, verified by a test. Propose the matching P1 requirement or rationale change.

A-13: P2's hand-offs to P3 (OI2-04, OI2-05, K-CC-POL) are not recorded in P1

A-13	Medium
Repo	P1 (seam with P2 and P3)
Where	P1 docs/01-system-description.md:36 'Software signing keys | Defined in P2 and P3 | ... | Candidate for the FC: PX4 Bootloader Secure Boot (Ed25519)'
Finding	P2 defines five signing keys (K-CC-ROOT, K-CC-OS, K-CC-POL, K-FC-FW, K-FC-REC) and hands P3 three obligations: OI2-04, a PKCS#11 Ed25519 signer, without which SR-034 cannot hold for K-FC-FW; OI2-05, MSS-authorized epoch commit; and K-CC-POL custody. None appears in P1: the key table still calls PX4 secure boot a 'candidate', and P1's open items list only OI-01 to OI-03 and DD-06 (P1 README.md:11-19). P3, which plans from P1, inherits these only through P2's docs.
Evidence	[verified] grep of P1 main for OI2, PKCS#11, epoch and K-CC-POL: no match.
Proposed fix	Propose a P1 change, as P2 did for changes 1 to 5: record P2's key table in P1 docs/01 §1.3 and add OI2-04 and OI2-05 (and A-01's removal case) as P1 open items owned by P3.

A-14: No path to put a new or MSS-generated key into a fielded CC's TPM (SR-036)

A-14	Medium
Repo	P2, P1 (seam with P3)
Where	P2 docs/03-measured-boot-sealing.md:3 'sealed once'; docs/03:73 'Re-sealing on the CC during an update was rejected. It needs the keys in the clear on a running system'; P1 data/requirements.csv:37 SR-036 'The MSS shall provision a new MAVLink signing key to the FC, CC and GCS after any loss of physical custody of a key holder'; P1 docs/01-system-description.md:35 WireGuard keys 'Generated by the MSS and loaded over wired links'
Finding	SR-036 requires a new MAVLink key to reach the CC after any loss of custody, and P1 has the MSS generate the CC's WireGuard key and load it. P2 seals once at the factory (emulated by the harness creating the key next to the TPM, tests/ve07/provision.py:183-191) and gives no way to put a new or MSS-generated key into a fielded CC's TPM without having it in clear on the CC. An MSS-wrapped key imported into the TPM would do this. No repo owns that path.
Evidence	[verified] P2 docs/03, docs/05 and tests/ve07/provision.py; P1 requirements and docs as cited.
Proposed fix	Define the provisioning and re-key path, for example an MSS-side duplication of each key to the CC's parent public key followed by TPM2_Import on the CC, so no plaintext key exists on the CC. Assign it (P2 or P3), and add a test case for re-keying under SR-036.

A-18: TPM reset only at power-on conflicts with the update flow's 'Reboot'; emulation hides it

A-18	Medium
Repo	P2 (seam with P3)
Where	docs/02-cc-boot-chain.md:99 DD2-17 'With power-on reset only, a warm reboot keeps the old PCRs, so the next boot's measurements do not match and unseal fails closed until a power cycle'; docs/03-measured-boot-sealing.md:69 update step 3 'Reboot: the new chain unseals with the new policy'; cc/uboot/qemu.config:4 (reset on any failure)
Finding	On the DEBIX design the TPM is reset only at power-on, so after any warm reset, including the reboot in the update flow and the reset U-Boot does after a refused boot, the keys cannot unseal until the board is power-cycled. The update flow says 'Reboot' and expects the new chain to unseal. The emulated chain does not model this: QEMU resets the emulated TPM with the machine, so no VE-07 case can show it. The difference is missing from the emulation table (docs/02 §2.4).
Evidence	[analysis] DD2-17 as written; QEMU's TPM backend resets the TPM on machine reset (not demonstrated here).
Proposed fix	Make update step 3 a power cycle, or a reset that also resets the TPM, and tell P3's update acceptance (SR-007). Add the difference to the docs/02 §2.4 table and a hardware test case for unseal after a warm reset.

A-24: DD2-14's recovery key is neither built nor tested; TC-14 cannot fail as written

A-24	Medium
Repo	P2
Where	docs/04-fc-secure-boot.md:39 'K-FC-FW lost | Sign with K-FC-REC. The firmware build's TOC must name slot 1, which needs a P2 variant of toc.c'; docs/04:61 DD2-14; data/testcases.csv:15 TC-14 pass criteria 'Each step returns the board to the intended state'
Finding	DD2-14's recovery key is neither built nor tested. fc/build.sh sets only PUBLIC_KEY0, so the CI boot loader has no slot 1; the toc.c variant does not exist; and TC-14's procedure (USB re-upload, key change over SWD, stock boot loader) never signs with K-FC-REC. TC-14's criterion cannot fail as written, because 'the intended state' is not defined.
Evidence	[verified] fc/build.sh:29-30; P2 tree has no toc.c variant; data/testcases.csv:15
Proposed fix	Either build the slot-1 path (boot loader with PUBLIC_KEY1, a toc.c variant, an image signed with K-FC-REC) and add it to TC-11 and TC-14, or withdraw DD2-14 to 'SWD is the only recovery'. Rewrite TC-14's criterion per step: which image runs, which key slot the boot loader reports.

A-15: docs/06 says every CI item is 'proved', beyond what the tests show

A-15	Low
Repo	P2
Where	docs/06-verification.md:18 'Every CI item is now proved'; docs/06-verification.md:24 'the sealing policy logic: approved state only, signed approval, epoch anti-rollback, sealed objects bound to one TPM, encrypted unseal'
Finding	'Proved' overstates two of the five sealing properties. Epoch anti-rollback is shown only against an honest older chain (A-01). Encrypted unseal is shown, but the salt that SR-010.4 requires is not tested (A-04). 'Approved state only' holds only until the first unseal (A-02).
Evidence	A-01, A-02, A-04.
Proposed fix	State what each CI item shows and under which condition; link the open findings until they are closed.

A-16: U-Boot binaries are not bit-reproducible between CI and a pinned local host

A-16	Low
Repo	P2
Where	cc/build.sh:13-14 'Every download is checked against a pinned SHA-256'; cc/uboot/pin.txt:8-10 (fixed SOURCE_DATE_EPOCH)
Finding	A local build in p2-noble with every host package at its pin reproduces CI byte for byte for Image.gz, both initramfs images, both agents, mkenvimage and all six .config files. It does not reproduce the six U-Boot binaries or mkimage. The predicted PCRs 0, 1, 8 and 9 are identical, so the measured state reproduces. Local rebuilds are identical to each other across paths and times. The cause of the CI difference was not found. The CI runner image carries compilers outside the pins (gcc-12, gcc-14 in toolchain.txt). For the hardware phase, where HAB signs u-boot.itb, an independent rebuild should give the same bytes.
Evidence	[verified] experiments/results via repro.py, repro2.sh, repro3.sh: 12 same, 7 differ; PCRs same; local hash 7fd6e136... at two paths; CI ef5af6bd...
Proposed fix	Diff a CI-built and a local u-boot.bin (upload the CI binary as an artifact once) to find the input that differs, then pin or neutralize it. Until then, docs should say that U-Boot binaries are not yet bit-reproducible across hosts.

A-17: The VE-07 and VE-08 host package lists cannot coexist on one host

A-17	Low
Repo	P2
Where	tests/ve08/README.md:29 'cc/emu/install-host-packages.sh fc/host-packages.txt'; tests/ve07/README.md:14 'cc/emu/install-host-packages.sh'
Finding	On Ubuntu 24.04 the two pinned package lists conflict. Installing fc/host-packages.txt (gcc-multilib) removed gcc-aarch64-linux-gnu and gcc-13-aarch64-linux-gnu from p2-noble on 2026-10-13 at 00:19:42, after which cc/build.sh cannot build. CI never sees this because the two workflows run on separate runners. Someone following both READMEs on one machine is left with a broken VE-07 setup and no warning.
Evidence	[verified] p2-noble /var/log/apt/history.log: 'Remove: gcc-13-aarch64-linux-gnu:amd64 (...), gcc-aarch64-linux-gnu:amd64 (...)' in the fc install; this audit reinstalled the VE-07 pins (experiments/setup.sh).

A-17	Low
Proposed fix	Say so in both READMEs and in docs/references.md, and give the order to restore. Or run TC-11's host checker in a container so the two sets never share a host.

A-19: docs/01 names P1 commit 83c6d2d; the snapshot is at e539aa6

A-19	Low
Repo	P2
Where	docs/01-scope-and-trace.md:3 'Every other P2 statement traces to P1 at commit 83c6d2d: baseline 1.0 plus change 1'
Finding	The snapshot is pinned to e539aa6 (data/p1/pin.txt:4; README.md:18), which includes changes 2 to 5.
Evidence	[verified]
Proposed fix	Say e539aa6, or point to data/p1/pin.txt instead of naming a commit.

A-20: Evidence column: folder in docs/06, file in the CSV

A-20	Low
Repo	P2
Where	docs/06-verification.md:61 'A test case's evidence column points at that folder'
Finding	data/testcases.csv cites .../summary.md, a file. The validator accepts either (A-07).
Evidence	[verified] data/testcases.csv:2-12
Proposed fix	Pick one form and enforce it (with A-07).

A-21: tests/ve07/README: stale variant count; local runs written into evidence/ve07-ci/

A-21	Low
Repo	P2
Where	tests/ve07/README.md:15 'U-Boot hardened, stock and two control variants'; tests/ve07/README.md:16 '--out evidence/ve07-ci/$(date -u +%Y%m%d)-$(git rev-parse --short HEAD)'
Finding	cc/build.sh builds six variants, including the two TC-06 test builds. The documented local command writes a local run into evidence/ve07-ci/, the folder whose name means a CI artifact (docs/06:66). Its summary says 'local', but the folder name does not.
Evidence	[verified] cc/build.sh:3-9, 52
Proposed fix	Say six variants. Point the local command at a scratch folder, or at evidence/ve07-local/.

A-22: docs/05 says PX4's test key is used only in TC-13; TC-11 uses it too

A-22	Low
Repo	P2
Where	docs/05-key-management.md:28 'P2 uses it only as the wrong-key case (TC-13)'
Finding	TC-11 also uses PX4's public test key as its wrong key (fc/build.sh:39-40; docs/04:54).
Evidence	[verified]
Proposed fix	Say TC-11 and TC-13.

A-23: docs/03 says TC-05 to TC-10 reproduced all prototype findings; two are not

A-23	Low
Repo	P2
Where	docs/03-measured-boot-sealing.md:85 'They are not evidence; TC-05 to TC-10 have since reproduced them'
Finding	Two of the six prototype findings are not reproduced by any test case. 'It unsealed for the next epoch before commit': TC-08b unseals release 2 only after the commit. 'swtpm 0.7.3 rejected an RSA-3072 public key for TPM2_LoadExternal' is not a test case at all.
Evidence	[verified] tests/ve07/boot_cases.py:799-811

A-23	Low
Proposed fix	Name which findings each test case reproduces, and mark the two others as prototype-only.

A-25: The parent-name check trusts a file on unprotected storage; its purpose is untested

A-25	Low
Repo	P2
Where	docs/03-measured-boot-sealing.md:31 'Its name is recorded at provisioning, and the unseal agent refuses any other key'; cc/initramfs/p2-unseal.c:205-209
Finding	The agent compares the parent's name with parent.name, which lives on the same unsigned, unmeasured CC storage as everything else the agent reads (p2-unseal.c:10). Anyone who can substitute the parent on the bus can also rewrite that file on a captured vehicle, so the check holds only while storage is intact. Removing the check entirely leaves the keys refused (at Load). TC-07a goes red only because the refusal moves to a different step; nothing tests what the check is for.
Evidence	M08 (experiments/results/M08-noparentcheck.txt): TC-07a fails on the step label; keys refused at Load; TC-07b unchanged.
Proposed fix	Either carry the expected parent name somewhere measured or signed, or state in docs/03 §3.2 and §3.6 that the check guards only against an interposer on an intact CC, and leave active interposers to P5 as §3.6 already does.

A-26: CI path filters miss files the harnesses import or run

A-26	Low
Repo	P2
Where	.github/workflows/ve08-host.yml:51-53 (paths); .github/workflows/ve07-emu.yml:7-9 (paths)
Finding	ve08-host does not run when tests/ve07/boot_cases.py changes, though image_cases.py imports its provenance helpers (tests/ve08/image_cases.py:47-48). ve07-emu runs tools/check_no_secrets.py on its evidence but does not run when that file changes. Neither runs when data/testcases.csv changes. Prior finding P2-06, narrowed.
Evidence	[verified]
Proposed fix	Add the imported and invoked files to each path filter, or drop the filters.

A-27: The secret scanner's fallback flags its own compiled test (2026-10-09 P2-13)

A-27	Low
Repo	P2
Where	docs/05-key-management.md:25 '`tools/check_no_secrets.py` scans every tracked and untracked, non-ignored file'; tools/check_no_secrets.py:38-39
Finding	Outside a git checkout the fallback scans everything except .git. After the unit tests run, it reports the compiled test file as a PEM and a PX4 private key (CPython folds the split strings back together), and it scans a local keys/ folder that .gitignore excludes. Prior finding P2-13 is still open.
Evidence	D4: on a git-archive copy, after python -m unittest: 'tests__pycache__\test_check_no_secrets.cpython-312.pyc: PEM private key', '...: PX4 secure-boot private key'.
Proposed fix	In the fallback, skip __pycache__, *.pyc and the .gitignore patterns. In the test, build the patterns at run time (for example from bytes.fromhex) so they are not folded.

A-28: Stale reference link, misdescribed snapshot mechanism, unexplained U-Boot log lines

A-28	Low
Repo	P2
Where	docs/references.md:15 U-Boot link 'https://source.denx.de/u-boot/u-boot'; docs/references.md:75 'apt-get update --snapshot'
Finding	Three items from 2026-10-09 are still open. P2-19: the link is stale and no note says v2026.10 exists. P2-20: install-host-packages.sh writes a deb822 source for snapshot.ubuntu.com and does not use `apt-get update --snapshot`. P2-21: every U-Boot log starts with 'Bloblist at 0 not found' and 'alloc space exhausted', and nothing explains them.
Evidence	[verified] cc/emu/install-host-packages.sh:17-25; every TC log in evidence/ve07-ci/20261013-d9149f6/
Proposed fix	As proposed on 2026-10-09 (P2-19 to P2-21).

A-29: Assumptions in P1 and P2 have no owner or retiring test

A-29	Low
Repo	P1, P2
Where	P1 docs/03-assumptions.md:3 'If one is invalidated, everything that cites it is re-examined'; P2 docs/01-scope-and-trace.md:43 'Test cases or open items retire them'
Finding	P1's 14 assumptions have IDs and rationales, but no owner and no way to be retired or checked. P2's six have no owner column either. A2-01 has no retiring test or open item (A-12), and A2-02 is retired 'on the module bought' with no test case.
Evidence	[verified]
Proposed fix	Add owner and 'retired by' columns to both tables.

A-30: P7 is assigned work in P1 but not defined or allocated

A-30	Low
Repo	P1
Where	P1 README.md:5 'P1 is the anchor for five follow-on repos, P2–P6'; P1 docs/04-architecture.md:215 DD-04 'P4 builds it and P7 fuzzes it'
Finding	P1 assigns fuzzing of the CC command filter to a P7 it does not otherwise define. No requirement or verification event is allocated to P7, and the memory-safe-language commitment in DD-04 has no requirement. The portfolio profile describes P7 as optional fuzzing of 'a MAVLink or CAN parser'. Prior findings P1-03 and P1-04 are still open.
Evidence	[verified] P1 main; samyoon727ca/README.md:13
Proposed fix	As proposed on 2026-10-09: name P7 in P1 with its scope and an owning requirement, or move the fuzzing commitment to P4.

A-31: P1 findings of 2026-10-09 still open (P1-01, -02, -06 to -15)

A-31	Low
Repo	P1
Where	Several, see observation
Finding	Open from 2026-10-09, re-checked on e539aa6. P1-01: brief/pdr-brief.md:29 'every number here is generated' (numbers are typed). P1-02: README.md:76-87 attributes drift and SysML checks to validate_trace.py. P1-07: docs/05-threat-model.md:108 THR-016/019 row omits SR-036 and P3. P1-08: README.md:24 'all 40 data-flow-diagram elements' (46 elements, 40 STRIDE elements). P1-09: the committed brief PDF is never compared with CI's. P1-10: python, java and node float by major version in validate.yml. P1-11: brief/package-lock.json:1256-1258 pins deprecated @xmldom/xmldom 0.9.10. P1-12: docs/references.md:32-42 rows without versions. P1-13: data/README.md omits gen_sysml.py. P1-14: model/sysml/README.md:30 'Java 21 only'. P1-15: README.md:15 'Closed by VE-02' under 'Still open'. P1-06 (README evidence-table 'Planned' column) also stands.
Evidence	[verified] grep of P1 main for each cited line
Proposed fix	As proposed on 2026-10-09; each item's fix is in _work/phase12/results/audit-p1.json.

A-32: Several P1 verification events have criteria that cannot fail

A-32	Low
Repo	P1
Where	P1 data/verification.csv:3 VE-02, :6 VE-05, :7 VE-06, :9 VE-08, :10 VE-09
Finding	Pass criteria that cannot fail or are underspecified. VE-02's procedure measures post-reboot replay, but its criteria say nothing about replay. VE-05's 'unreadable' has no test behind it. VE-06's 'follows the defined order' cites no definition. VE-08's 'Only the signed image starts PX4' does not name the key, and its procedure omits a wrong-key image. VE-09 bundles six requirements and its watchdog criterion has no time bound. Method fit: SR-003 (signing) and SR-034 (HSM non-export) are verified by Demonstration only.
Evidence	[verified] P1 data/verification.csv
Proposed fix	Give each criterion an observable outcome that can fail (for VE-05, a decryption with the wrong key that must fail; for VE-02, a stated replay result). Split VE-09 per requirement, or record per-requirement results inside it.

A-33: P1 has no .gitattributes; Windows checkouts get CRLF

A-33	Low
Repo	P1
Where	P1 repo root (no .gitattributes)
Finding	P1 has no .gitattributes. On this machine (core.autocrlf=true), git archive and checkouts write CRLF, and P1's own render_views.py writes CRLF on Windows. P2 fixed the same problem for itself (P2-12).
Evidence	[verified] a git archive export of P1 main had CRLF CSVs; the blobs themselves are LF and identical to P2's snapshot.
Proposed fix	Add '* text=auto eol=lf' as P2 did.

A-34: The harness's 'modified' flag ignores tools/, which it imports

A-34	Low
Repo	P2
Where	tests/ve07/boot_cases.py:943 git_state 'cc_or_tests_modified' (status of cc and tests only)
Finding	The harness imports tools/predict_pcrs.py and tools/approve_policy.py, but the 'modified' flag in summary.json ignores tools/. A local run with a modified predictor would report 'cc/ or tests/ modified: false'. CI is unaffected because its checkout is clean.
Evidence	[verified]
Proposed fix	Include tools/ (and fc/ for VE-08) in the status check.

A-35: Checked and sound: evidence, snapshot, baseline, most mutations

A-35	Info
Repo	P2
Where	README.md:14; evidence/README.md:5-12; data/p1/pin.txt
Finding	Checked and sound. All six evidence folders are byte-identical to their GitHub artifacts. The P1 snapshot equals P1 at e539aa6, which is P1 main. validate_trace.py, render_views.py --check and check_no_secrets.py pass on main. The full VE-07 harness passes 31 of 31 runs locally. Most single-point mutations turn a test red: M01, M02, M03, M04a, M05, M06, plus the shared-constant mutations of §2.2 that have a negative case.
Evidence	experiments/results/ and report §2, §4
Proposed fix	None.

Appendix: glossary and sources

Glossary

Term	Meaning in this document
A/B layout	Two copies of the boot chain or image on storage, so a device can fall back to the previous one if a new one fails
authPolicy	The policy digest a TPM object requires before it can be used (“Policy sessions” in §2.2)
Bank (PCR)	One set of PCRs for one hash algorithm; P2's policy uses the SHA-256 bank
BL31	The runtime stage of Arm Trusted Firmware (TF-A), loaded with U-Boot in `u-boot.itb` on the i.MX 8M Plus
Boot ROM	Code in the SoC's mask ROM that runs first at power-on and cannot be changed
`bootm`	The U-Boot command that boots an image; with measured boot it also extends the PCRs
C2	Command and control of the vehicle
CC	Companion computer (P1 CMP-CC): the vehicle's Linux computer; in P2 a DEBIX Model A
CSF, CST	Command Sequence File and NXP's Code-Signing Tool that produces it, for HAB (“NXP High Assurance Boot (HAB) on the i.MX 8M Plus” in §2.2)
Devicetree	A data structure describing hardware to the boot loader and kernel; also the binary format FIT images use
dm-verity	A Linux mechanism that checks every block of a read-only filesystem against a hash tree (P4's SR-006)
ECSPI	The i.MX SPI controller; the DEBIX's TPM would sit on ECSPI2
Ed25519	The signature scheme PX4's secure boot uses: 32-byte public keys, 64-byte signatures
Epoch	P2's anti-rollback number, kept as bits in a TPM NV index (“NV indices and the bit-field epoch” in §2.2)
ESAPI	The Enhanced System API of the TPM software stack (tpm2-tss), used by `p2-unseal`
Event log	The record of every PCR extend, which replays to the PCR values (“Measured boot: PCRs, extend and the event log” in §2.2)
Extend	The only way to change a PCR: new = H(old ‖ digest)
FC	Flight controller (P1 CMP-FC): PX4 on a Pixhawk 6X
FIT	Flattened Image Tree, U-Boot's signed multi-image format (“U-Boot FIT images and configuration signing” in §2.2)
Fuse (eFuse, OTP)	One-time-programmable bits in the SoC; burning one is permanent
GCS	Ground control station (P1 CMP-GCS)
HAB	High Assurance Boot, NXP's boot-ROM verification (“NXP High Assurance Boot (HAB) on the i.MX 8M Plus” in §2.2)
HSM	Hardware security module: a device that holds private keys and signs without exporting them
initramfs	The small root filesystem the kernel starts first; in P2 it holds `p2-unseal`
MAVLink	The command and telemetry protocol between GCS, CC and FC; MAVLink 2 can sign frames
MSS	Maintenance and support station (P1 CMP-MSS): builds, signs and provisions
Name (TPM)	The digest of a TPM object's public area, used to identify it in policies
NV index	A small area of TPM non-volatile memory with its own type and authorization
PCR	Platform Configuration Register, a TPM register changed only by extend (“Measured boot: PCRs, extend and the event log” in §2.2)
PKCS#11	The standard API through which software asks an HSM to sign
Policy session	A TPM session in which policy commands build a digest that must match an object's authPolicy (“Policy sessions” in §2.2)
PolicyAuthorize	A policy command that accepts any policy signed by a named key (“PolicyAuthorize: approval by signature” in §2.2)
policyRef	A string signed together with an approved policy, naming its use (`p2/cc-keys`)
QEMU `virt`	QEMU's generic Arm machine, used for the emulated CC
Salted session	A TPM session whose key derives from a secret encrypted to a TPM-resident key (“Salted sessions and response encryption” in §2.2)
Sealing	Storing a secret under a TPM so that it is released only when a policy is satisfied (“TPM objects and sealing” in §2.2)
SITL	PX4 software-in-the-loop simulation (P3's harness)

Term	Meaning in this document
SoC	System-on-chip; the i.MX 8M Plus on the CC
SPL	U-Boot's secondary program loader, the first stage the i.MX boot ROM loads
SRK	Super Root Key; HAB's table of up to four root public keys, whose hash is fused
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
swtpm	A software TPM 2.0 emulator; P2's CI uses swtpm 0.7.3
SWD	Serial Wire Debug, the STM32's debug port, used to flash the boot loader
TOC	The table of contents in a PX4 secure-boot image (“PX4 Bootloader Secure Boot: the TOC and Ed25519” in §2.2)
TPM	Trusted Platform Module, TPM 2.0 in P2
U-Boot	The open-source boot loader P2 uses, release `v2026.07`
WireGuard	The VPN tunnel between GCS and CC (P1 DD-02)

Sources

The two repositories, at the commits this document was built from (see Built from):

- P1: https://github.com/samyoon727ca/uas-sec-p1-architecture, data at `e539aa638d3e44c1982341e927d974b4396e0576`. Documents cited: `README.md`, `data/README.md`, `docs/01-system-description.md` to `docs/08-verification-plan.md`, `docs/references.md`, `.github/workflows/validate.yml`.
- P2: https://github.com/samyoon727ca/uas-sec-p2-verified-boot at `db31c1451c7af859ae08cc2d4b073980cc666a1a`. Documents cited: `README.md`, `docs/01-scope-and-trace.md` to `docs/06-verification.md`, `docs/references.md`, `evidence/README.md`, `tests/ve07/README.md`, `tests/ve08/README.md`, and the source files cited by line.

External sources, as P2 pins and cites them (P2 `docs/references.md`; versions and hashes are P2's):

Tag	Source	Version P2 pins
[UBOOT]	U-Boot release tarball `u-boot-2026.07.tar.bz2`	`v2026.07`, SHA-256 `78e8bfc382fe388f9b55aa1daf8c563522a037779b5d4c349d1415e381f1243e`
[UBOOT-SIG]	U-Boot `doc/usage/fit/signature.rst`, `doc/usage/fit/sign-configs.rst`	`v2026.07`
[UBOOT-MB]	U-Boot `doc/usage/measured_boot.rst`	`v2026.07`
[UBOOT-HAB]	U-Boot `doc/imx/habv4/`	`v2026.07`
[NXP-CST]	NXP Code-Signing Tool User's Guide	Rev. 3.3.1
[NXP-FUSE]	NXP `uboot-imx` `doc/imx/habv4/guides/mx8m_secure_boot.txt`	commit `bb41975a`
[TCG-P1], [TCG-P2]	TCG TPM 2.0 Library, Part 1 (Architecture) and Part 2 (Structures)	Revision 01.59
[SP800-57]	NIST SP 800-57 Part 1 Rev. 5	May 2020
[PX4-SECBOOT], [PX4-BL]	PX4 Guide: Bootloader Secure Boot; PX4-Autopilot boot-loader source	`v1.18.0-rc1`, commit `fca3df86`
[SWTPM]	swtpm	0.7.3 (Ubuntu `0.7.3-0ubuntu5.24.04.1`)
[TPM2-TSS], [TPM2-TOOLS]	tpm2-tss ESAPI; tpm2-tools	4.0.1; 5.6
[QEMU]	QEMU `hw/arm/virt.c`, `hw/core/sysbus-fdt.c`	`v8.2.2`